MACHINE LEARNING PORTFOLIO · MSC DATA SCIENCE · ARDEN UNIVERSITY BERLIN

Product Segmentation and Performance Clustering for an E-Commerce Fragrance Marketplace

An end-to-end unsupervised machine learning investigation comparing K-Means, Ward hierarchical clustering, DBSCAN, HDBSCAN and the TabClustPFN foundation model across 1,000 marketplace listings, translated into evidence-based actions for catalogue strategy.

STUDENT ID	**25199053**
MODULE	COM7022 Machine Learning · Assignment [5444]
ASSESSMENT	Portfolio · Task 1 Practical Project · Task 2 Critical Report
WORD COUNT	3,319 words of body text (Task 1: 2,372 · Task 2: 947); captions, tables, figures, code screenshots, references and appendix excluded
DATASET	Sales-[5444].csv · 1,000 listings × 10 attributes
REPOSITORY	https://github.com/kvmmn/_COM7022_MachineLearning
REVIEW PORTAL	https://kvmmn.github.io/_COM7022_MachineLearning/

CORE REVENUE ENGINE	INVENTORY RISK	MODEL STABILITY	PCA VARIANCE
92.3%	**31.3%**	**0.932**	**78.7%**
of units sold come from 24.3% of listings (Cluster 2)	of listings sit in the low-stock tail (Cluster 3)	K-Means bootstrap ARI over 30 resamples	retained by the first two components

Every figure, table and code screenshot carries a caption that names and links its exact file in the GitHub repository.

Contents

1. Introduction 3

1.1 Business Context 3

1.2 The Dataset 3

1.3 Problem Statement 3

1.4 Project Objectives 3

1.5 Methodological Approach 3

TASK 1 · PRACTICAL MACHINE LEARNING INVESTIGATION

2. Data Preparation 5

2.1 Data Quality Audit 5

2.2 Cleaning Strategy 6

2.3 Transformation, Scaling and Categorical Variables 9

2.4 Feature Selection 11

3. Model Selection 11

3.1 Clustering in Brief 11

3.2 Candidate Techniques 12

3.3 Dimensionality Reduction with PCA 12

4. Model Implementation and Evaluation 13

4.1 K-Means 13

4.2 Hierarchical Clustering 14

4.3 DBSCAN 15

4.4 HDBSCAN 16

4.5 TabClustPFN 17

4.6 Evaluation Framework 17

4.7 Comparative Results 18

5. Cluster Characteristics 19

TASK 2 · CRITICAL REPORT FOR EXECUTIVE STAKEHOLDERS

6. Executive Summary 22

7. Critical Evaluation 22

7.1 Suitability of the Methods and Design Decisions 22

7.2 Comparative Performance: Strengths and Limitations 22

7.3 Why K-Means Is the Right Choice for E-Commerce 23

8. Recommendations for Business Managers 23

9. Limitations 24

10. Conclusion and Future Work 24

References 25

Appendix A: Reproducible Code 26

1. Introduction

1.1 Business Context

This portfolio is written as a newly appointed Machine Learning Analyst at a rapidly growing multi-seller e-commerce marketplace. Independent merchants list competing offers for the same products, so the catalogue grows faster than any team can curate by hand. Decisions on which listings to promote, replenish or retire now require a scalable, data-driven view.

1.2 The Dataset

The analysis uses the company extract `Sales-[5444].csv`: **1,000 listings described by 10 attributes** covering product identity (`Item_Name`, `details`, `Catatory`), pricing (`price`, `priceWithCurrency`), availability and traction (`available`, `text`, `sold`), maintenance history (`lastUpdated`) and dispatch origin (`itemLocation`). The catalogue is a men's fragrance category, dominated by US-based sellers (909 listings) with a smaller cross-border segment led by Hong Kong (52). Critically, each row is a **seller listing rather than a unique product**: the same fragrance can appear several times at different prices and stock positions, so the resulting segments describe commercial offers, not manufacturers.

1.3 Problem Statement

Three obstacles prevent the company from managing this catalogue strategically. First, **scale**: manual review cannot keep pace with listing growth. Second, **data quality**: prices and sales are partly stored as free text, category labels are inconsistent and the platform censors stock above ten units. Third, **the absence of labels**: no field states whether a listing is a best-seller, a premium niche item or an inventory risk, which rules out supervised learning. The business question is therefore one of *discovery*: what natural commercial groups exist in the catalogue, and how should each be managed?

1.4 Project Objectives

- **O1 — Data readiness:** audit, clean and transform the dataset with every intervention evidenced and reproducible.
- **O2 — Segmentation:** apply K-Means, Hierarchical (agglomerative) clustering, DBSCAN, the more recent HDBSCAN and the TabClustPFN foundation model to group listings by product characteristics and sales performance.
- **O3 — Rigorous evaluation:** compare models with complementary internal, stability and sensitivity measures rather than a single score.
- **O4 — Model justification:** select the model that is technically strongest *and* operationally suited to e-commerce.
- **O5 — Business translation:** convert segments into evidence-based actions for non-technical decision-makers.

1.5 Methodological Approach

The investigation follows a structured machine learning lifecycle (Figure 1). Each stage is implemented in a single Google Colab notebook with fixed random seeds, and every analytical decision is logged so that results can be regenerated end to end. The notebook was re-executed from a clean state before this report was written and reproduced every reported figure.

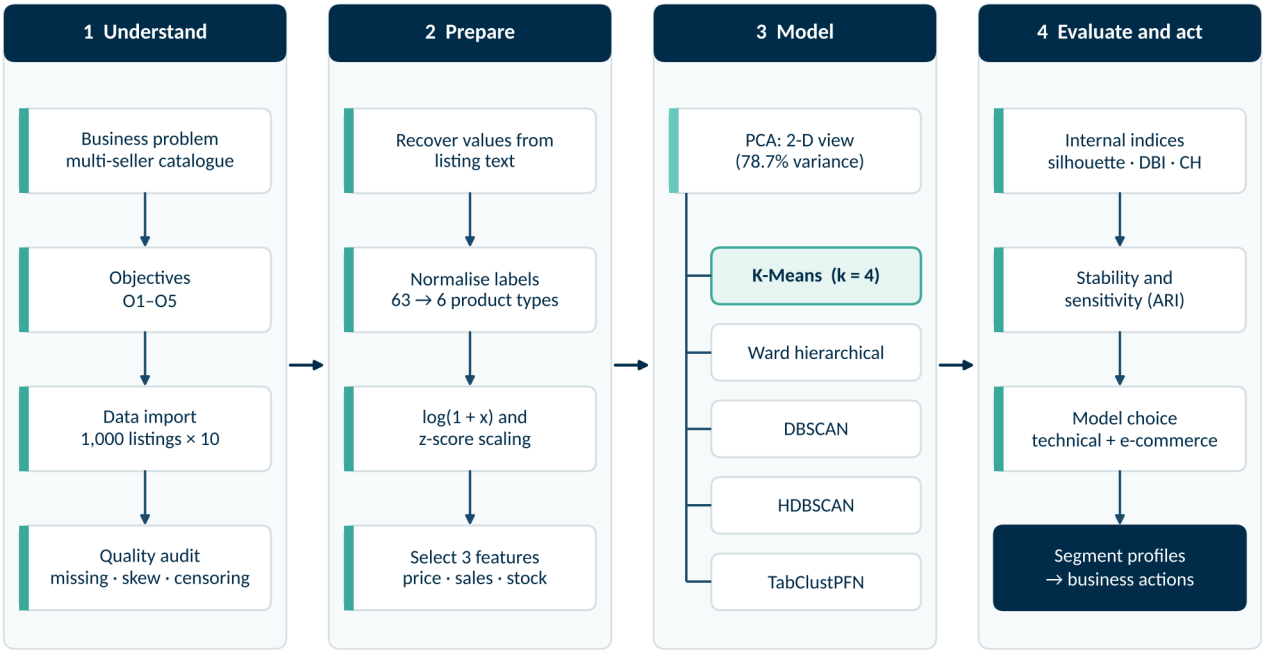


Figure 1. End-to-end methodological workflow, from business problem to recommended actions.
Source: COM7022 - Assignment/output/figures/fig15_workflow.png

TASK 1

Practical Machine Learning Investigation

2,372 words of body text

2. Data Preparation

2.1 Data Quality Audit

A systematic audit preceded any modelling. The file contains **no full-row duplicates**, although 34 listings repeat an identical description, consistent with multi-seller offers rather than data-entry error. Missing values affect eight columns (Table 1, Figure 2); the largest gaps are in `available` (111 rows, 11.1%) and `lastUpdated` (53 rows, 5.3%).

Descriptive profiling exposed four structural problems that would distort any distance-based algorithm:

- **Extreme skew.** Units sold range from 1 to 54,052 against a median of 49.5 (skewness 9.70); price is also right-skewed (2.04). Euclidean distances on raw values would be dominated by a handful of viral listings (Figure 3).
- **Censored stock.** The platform displays "More than 10 available" for 299 listings and stores the value as 10, so recorded stock is a ceiling, not a measurement.
- **Label fragmentation.** `Catatory` holds 63 distinct strings that are largely spelling and capitalisation variants of a few fragrance types.
- **Mixed currency.** Three prices are quoted in Canadian dollars.

Table 1. Missing values per attribute in the raw extract (n = 1,000).

Attribute	Missing	Missing %
available	111	11.1
lastUpdated	53	5.3
sold	8	0.8
price	5	0.5
text	3	0.3
Catatory	3	0.3
priceWithCurrency	1	0.1
Item_Name	1	0.1

Source: COM7022 - Assignment/output/tables/table01_missing_values.csv

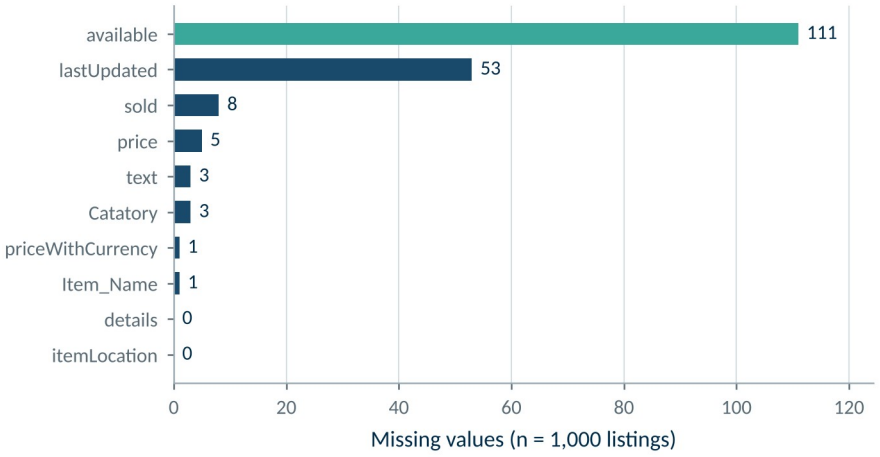


Figure 2. Missing values by attribute in the raw extract.
Source: COM7022 - Assignment/output/figures/fig01_missing_values.png

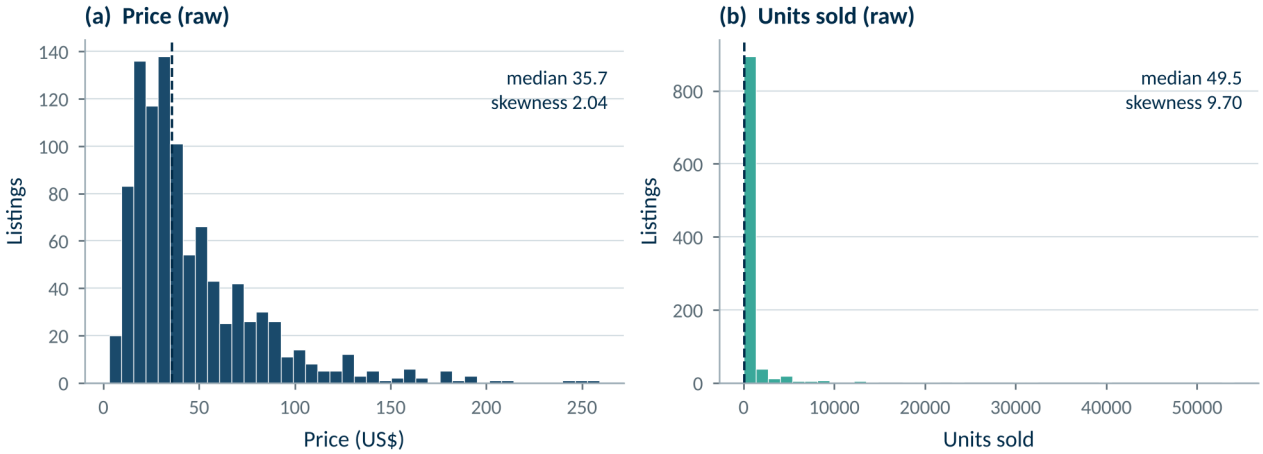


Figure 3. Raw distributions of price and units sold, showing the extreme right skew.
Source: COM7022 - Assignment/output/figures/fig02_raw_distributions.png

2.2 Cleaning Strategy

Preparation followed a single principle: **recover before you impute**. Where a value was absent, it was first reconstructed from another field of the same listing; statistical imputation was used only where the cause of the gap was understood, and it was always flagged. Every step was recorded in an execution log (Table 2) and the overall flow is summarised in Figure 4.

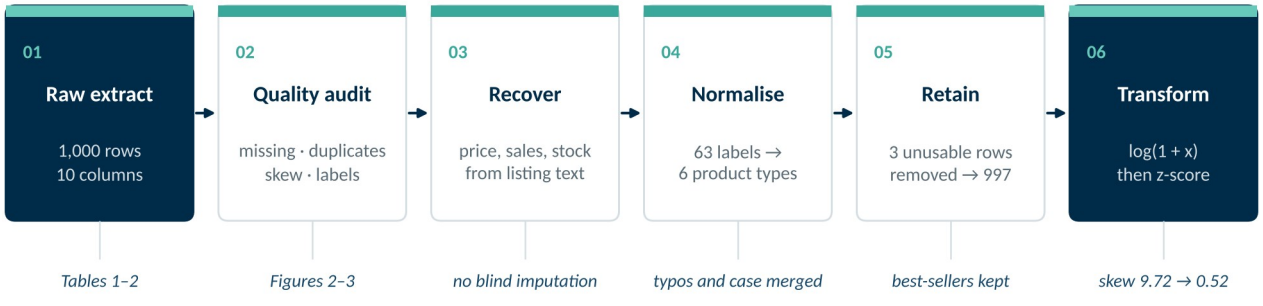


Figure 4. Data preparation pipeline from the raw extract to model-ready features.
Source: COM7022 - Assignment/output/figures/fig16_data_prep_flow.png

- **Price and sales recovery.** All five missing prices were restored from `priceWithCurrency`, which matched the numeric field exactly in all 994 rows holding both. Two sales figures were parsed from `text`; three listings showing stock but no sales were set to zero; Canadian prices were converted at a documented rate.

- **Stock reconstruction.** Availability was rebuilt from listing wording: "More than 10" and any explicit value above ten were coded as 11 (*well stocked*), "Last One" as 1, and the 62 "Limited quantity" or lot listings received the median with an indicator column, so their influence could be audited.
- **Category normalisation.** The 63 raw labels were consolidated into six coherent product types (Eau de Toilette, Eau de Parfum, Cologne, Parfum/Extrait, Generic fragrance, Other).
- **Record retention.** Only three listings with neither sales nor stock information were removed, leaving **997 verified listings**. No commercially extreme listing was discarded, because best-sellers are precisely the observations the business must understand.

Table 2. Execution log of every cleaning and transformation step.

Pre-processing step	Rows affected	Rationale
Recover price from priceWithCurrency	5	Exact same value, only stored in text form
Convert CAD prices to USD	3	Fixed rate 0.73, documented approximation
Recover sold from text	2	Exact value from the listing wording
Set sold = 0 where text shows stock but no sales	3	Listing wording omits 'sold' when nothing has sold
Drop rows with no sales or stock information	3	Cannot be placed on the sales-performance axis
Stock: 'More than 10' coded as 11	299	Censored value, not exact
Stock: explicit values above 10 capped at 11	115	Same scale as the censored rows
Stock: 'Last One' coded as 1	47	Wording means one unit left
Stock: median imputation	62	Limited quantity / lot wording; flagged in stock_imputed
Collapse category labels into 6 product types	994	Case, spelling and wording variants
Missing brand set to 'Unbranded'	1	Single row
Log-transform price and sold	997	Outliers kept; log scale limits their pull on distances

Source: COM7022 - Assignment/output/tables/table03_cleaning_log.csv

COM7022_Clustering_25199053.ipynb | **Missing-value audit**

```
# Missing values per column, as counts and percentages
missing = pd.DataFrame({
    "missing_count": raw.isna().sum(),
    "missing_pct": (raw.isna().mean() * 100).round(1),
})
missing = missing.sort_values("missing_count", ascending=False)
print("Full-row duplicates:", raw.duplicated().sum())
print("Duplicate listing descriptions:", raw["details"].duplicated().sum())
save_table(missing, "table01_missing_values.csv")
missing
```

Figure 5. Code screenshot: Missing-value and duplicate audit.

Source: COM7022 - Assignment/output/screenshots/code02_missing_values.png

COM7022_Clustering_25199053.ipynb | **Cleaning: recover price**

```python
df = raw.copy()
cleaning_log = []


def log_step(step, rows_affected, note):
    cleaning_log.append({"step": step, "rows_affected": int(rows_affected), "note": note})


# 3.1 Price: recover missing prices from priceWithCurrency (same listing, same number)
price_from_text = df["priceWithCurrency"].str.extract(r"\$([\d,]+\.?\d*)")[0]
price_from_text = price_from_text.str.replace(",", "").astype(float)
recovered = df["price"].isna() & price_from_text.notna()
df.loc[recovered, "price"] = price_from_text[recovered]
log_step("Recover price from priceWithCurrency", recovered.sum(), "Exact same value, only stored in text form")

# Check: where both exist, do they agree?
both = df["price"].notna() & price_from_text.notna()
print("Price disagreements where both exist:", (abs(df.loc[both, "price"] - price_from_text[both]) > 0.01).sum())

# 3.2 Currency: convert the three Canadian-dollar prices to USD
# Approximate May 2024 rate (1 CAD ~ 0.73 USD). Only 3 rows, so the exact rate barely matters.
CAD_TO_USD = 0.73
is_cad = df["priceWithCurrency"].fillna("").str.startswith("C")
df.loc[is_cad, "price"] = (df.loc[is_cad, "price"] * CAD_TO_USD).round(2)
log_step("Convert CAD prices to USD", is_cad.sum(), "Fixed rate 0.73, documented approximation")
```

Figure 6. Code screenshot: Recovery of price from priceWithCurrency and currency conversion.
Source: COM7022 - Assignment/output/screenshots/code03_price_recovery.png

COM7022_Clustering_25199053.ipynb | **Cleaning: recover units sold**

```python
# 3.3 Units sold: recover from the 'text' column ("... / 116 sold")
sold_from_text = df["text"].str.extract(r"([\d,]+) sold")[0].str.replace(",", "").astype(float)
recovered = df["sold"].isna() & sold_from_text.notna()
df.loc[recovered, "sold"] = sold_from_text[recovered]
log_step("Recover sold from text", recovered.sum(), "Exact value from the listing wording")

# Check the two sources agree everywhere else
both = df["sold"].notna() & sold_from_text.notna()
print("Sold disagreements where both exist:", (df.loc[both, "sold"] != sold_from_text[both]).sum())

# Listings whose text shows stock but no 'sold' part have not sold yet, so sold = 0
no_sales_yet = df["sold"].isna() & df["text"].notna() & ~df["text"].str.contains("sold", na=False)
df.loc[no_sales_yet, "sold"] = 0
log_step("Set sold = 0 where text shows stock but no sales", no_sales_yet.sum(), "Listing wording omits 'sold' when nothing has sold")

# Rows with no text and no sold value carry no sales or stock information at all
no_info = df["sold"].isna() & df["text"].isna()
print(df.loc[no_info, ["Item_Name", "price", "available", "text", "sold"]])
df = df[~no_info].copy()
log_step("Drop rows with no sales or stock information", no_info.sum(), "Cannot be placed on the sales-performance axis")
```

Figure 7. Code screenshot: Recovery of units sold from the listing text.
Source: COM7022 - Assignment/output/screenshots/code04_sold_recovery.png

COM7022_Clustering_25199053.ipynb | **Cleaning: stock level and censoring**

```python
# 3.4 Stock level: rebuild it from the listing wording so every row has a value.
# The marketplace hides exact numbers above 10 ("More than 10 available"), so I cap all
# stock values at 11, meaning "more than 10". This keeps the measure honest and comparable.
text = df["text"].fillna("")
stock = text.str.extract(r"^(\d+) available")[0].astype(float)
stock[text.str.startswith("More than")] = 11
stock[text.str.startswith("Last One")] = 1
capped_above_10 = (stock > 11).sum()
stock = stock.clip(upper=11)

# "Limited quantity available" and lot listings give no number: impute the median, but keep a flag
stock_imputed = stock.isna()
df["stock_imputed"] = stock_imputed.astype(int)
stock = stock.fillna(stock.median())
df["stock_level"] = stock

log_step("Stock: 'More than 10' coded as 11", text.str.startswith("More than").sum(), "Censored value, not exact")
log_step("Stock: explicit values above 10 capped at 11", capped_above_10, "Same scale as the censored rows")
log_step("Stock: 'Last One' coded as 1", text.str.startswith("Last One").sum(), "Wording means one unit left")
log_step("Stock: median imputation", stock_imputed.sum(), "Limited quantity / lot wording; flagged in stock_imputed")
print(df["stock_level"].value_counts().sort_index())
```

Figure 8. Code screenshot: Stock reconstruction and treatment of censoring.
Source: COM7022 - Assignment/output/screenshots/code05_stock_level.png

COM7022_Clustering_25199053.ipynb | **Cleaning: normalise product type**

```python
# 3.5 Product type: collapse the messy category labels into six groups


def group_category(label):
    if pd.isna(label):
        return "Other"
    s = str(label).strip().lower()
    if s in ["", "/", "y", "does not apply", "unknown", "various", "assorted"]:
        return "Other"
    if "," in s or "/" in s:
        return "Other"                          # listings that name several types at once
    if "toil" in s or s == "edt":
        return "Eau de Toilette"                # includes the misspelling 'toillette'
    if "eau de parfum" in s or "eau de perfume" in s:
        return "Eau de Parfum"
    if "parfum" in s or "extrait" in s or "elixir" in s:
        return "Parfum / Extrait"
    if "cologne" in s or s == "edc":
        return "Cologne"
    if "perfume" in s or "fragrance" in s:
        return "Generic fragrance"
    return "Other"                              # body sprays, oils, deodorants, gift sets


df["product_type"] = df["Catatory"].apply(group_category)
print("Raw labels:", df["Catatory"].nunique(), "-> groups:", df["product_type"].nunique())
print(df["product_type"].value_counts())
log_step("Collapse category labels into 6 product types", df["Catatory"].notna().sum(), "Case, spelling and wording variants")

# 3.6 Brand: tidy whitespace; a missing brand is treated as 'Unbranded'
df["brand"] = df["Item_Name"].fillna("Unbranded").str.strip()
log_step("Missing brand set to 'Unbranded'", df["Item_Name"].isna().sum(), "Single row")

# 3.7 Location: keep the country (last part of 'City, State, Country')
df["country"] = df["itemLocation"].str.split(",").str[-1].str.strip()
df["ships_from_us"] = (df["country"] == "United States").astype(int)

# 3.8 Last update: parse dates (two month formats) and turn them into 'days since update'
dates = pd.to_datetime(df["lastUpdated"].str.replace(r" P[DS]T", "", regex=True), format="mixed", errors="coerce")
df["days_since_update"] = (dates.max() - dates).dt.total_seconds() / 86400
print("Unparsed or missing dates:", dates.isna().sum(), "| latest update:", dates.max())

# 3.9 Bottle size in ounces, parsed from the description (used for profiling, not clustering)
size = df["details"].str.extract(r"(\d+(?:\.\d+)?)\s*(?:fl\.?\s*)?oz", flags=re.IGNORECASE)[0].astype(float)
df["size_oz"] = size.where(size.between(0.3, 8.5))   # ignore implausible parses
print("Bottle size found for", df["size_oz"].notna().sum(), "listings")
```

Figure 9. Code screenshot: Normalisation of 63 category labels into six product types.
Source: COM7022 - Assignment/output/screenshots/code06_product_type.png

2.3 Transformation, Scaling and Categorical Variables

Logarithmic transformation. A log(1 + x) transform was applied to price and units sold. It preserves zero values and rank order while compressing the long right tail, reducing the skewness of units sold from 9.72 to 0.52 and of price from 1.97 to 0.09 (Table 3, Figure 10).

Standardisation. Because all three algorithms rely on Euclidean distance, each feature was converted to a z-score so that it contributes equal variance:

$$z = \frac{x - \mu}{\sigma}$$

Min-max normalisation was evaluated as an alternative and rejected: it inflates the bounded 1–11 stock scale relative to the log-transformed measures and changed the partition substantially (ARI 0.49; Section 4.7).

Categorical variables. Brand (246 values), product type and country were deliberately excluded from the distance computation. One-hot encoding would add hundreds of sparse dimensions and allow single brands to dominate the geometry; these attributes were instead reserved for post-hoc cluster profiling.

Table 3. Descriptive statistics of the 997 cleaned listings before and after the log transform.

Variable	Mean	Std	Min	Median	Max	Skewness
price (US$)	46.07	34.75	3	35.62	259.09	1.97
units sold	771.63	3203.77	0	49	54,052	9.72
stock level	7.73	3.53	1	9	11	-0.57
log price	3.63	0.67	1.39	3.6	5.56	0.09
log units sold	4.29	2.08	0	3.91	10.9	0.52

Source: COM7022 - Assignment/output/tables/table04_descriptive_statistics_clean.csv

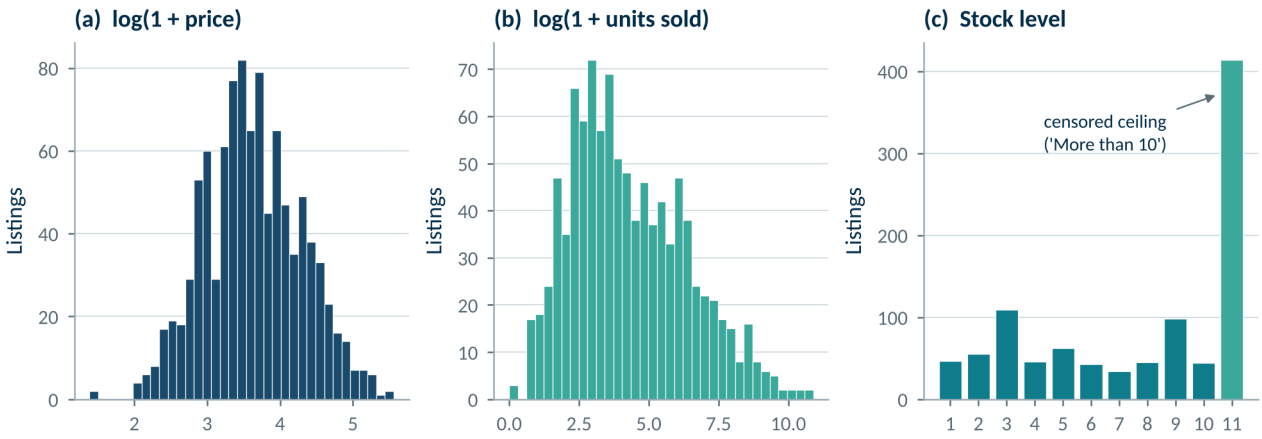


Figure 10. Feature distributions after the logarithmic transformation.

Source: COM7022 - Assignment/output/figures/fig03_transformed_distributions.png

COM7022_Clustering_25199053.ipynb | **Transformation: log(1 + x)**

```
# 3.10 Transform the skewed measures. log(1 + x) keeps zero sales valid (log of 0 is undefined).
df["log_price"] = np.log1p(df["price"])
df["log_sold"] = np.log1p(df["sold"])

skew_table = pd.DataFrame({
    "skew_before": [skew(df["price"]), skew(df["sold"])],
    "skew_after_log": [skew(df["log_price"]), skew(df["log_sold"])],
}, index=["price", "sold"]).round(2)
print(skew_table)

# Outliers: count them with the 1.5 x IQR rule on the raw scale, but keep them.
# Best-selling listings are exactly what the business wants to find, so removing them would be wrong.
q1, q3 = df["sold"].quantile([0.25, 0.75])
upper_fence = q3 + 1.5 * (q3 - q1)
print("Listings above the sold upper fence (" + str(round(upper_fence)) + "):", (df["sold"] > upper_fence).sum())
log_step("Log-transform price and sold", len(df), "Outliers kept; log scale limits their pull on distances")

cleaning_table = pd.DataFrame(cleaning_log)
save_table(cleaning_table, "table03_cleaning_log.csv")
print("Rows after cleaning:", len(df))
cleaning_table
```

Figure 11. Code screenshot: Logarithmic transformation of skewed measures.

Source: COM7022 - Assignment/output/screenshots/code07_log_transform.png

COM7022_Clustering_25199053.ipynb | **Scaling: StandardScaler**

```
FEATURES = ["log_price", "log_sold", "stock_level"]
scaler = StandardScaler()
X = scaler.fit_transform(df[FEATURES])
print("Feature matrix:", X.shape)
print("Means after scaling:", X.mean(axis=0).round(3))
print("Std after scaling:", X.std(axis=0).round(3))
```

Figure 12. Code screenshot: Z-score standardisation of the selected features.

Source: COM7022 - Assignment/output/screenshots/code08_scaling.png

2.4 Feature Selection

Candidate features were screened with Spearman rank correlation (Figure 13) and a systematic comparison of five feature sets (Table 4). The correlations are weak to moderate (price–sales −0.13, price–stock −0.20, sales–stock +0.35), confirming that each candidate captures a distinct commercial mechanism without problematic collinearity.

The final set comprises **log price, log units sold and stock level** (Set B), representing market position, demand and supply resilience. A two-feature baseline (Set A) achieves a marginally higher silhouette but ignores availability, which is central to the brief and to stock-out risk. Adding listing recency or bottle size reduced separation without adding actionable meaning: half of all bottles fall between 3.3 and 3.7 oz, and three-quarters of listings were updated within the past week.

Table 4. K-Means silhouette scores for five candidate feature sets (higher is better).

Feature set	Features	k = 2	k = 3	k = 4	k = 5	k = 6
A: price + sold	2	0.351	0.377	0.356	0.351	0.354
B: price + sold + stock	3	0.337	0.297	0.307	0.308	0.297
C: B + days since update	4	0.31	0.315	0.277	0.288	0.284
D: B + bottle size	4	0.274	0.308	0.271	0.272	0.272
E: B + days + size	5	0.256	0.259	0.292	0.255	0.258

Source: COM7022 - Assignment/output/tables/table05_feature_set_scan.csv

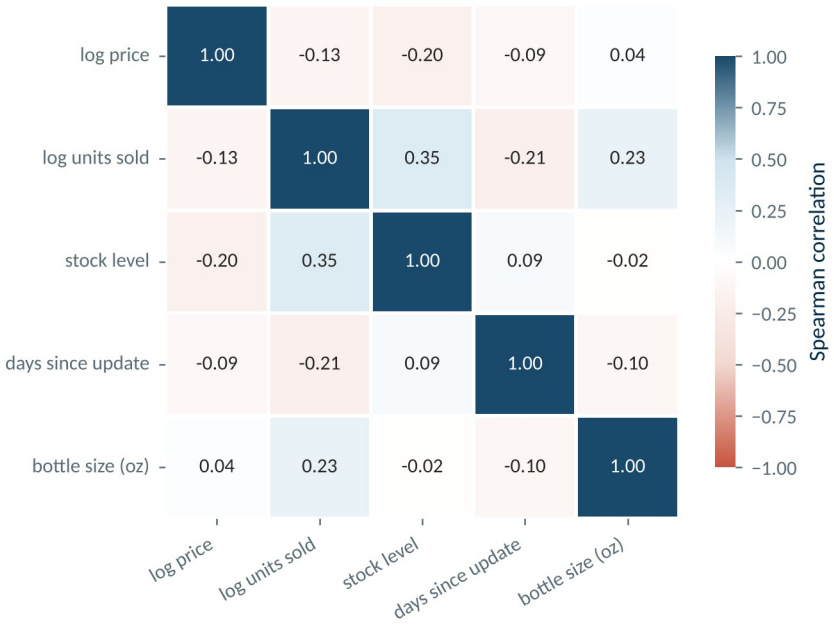


Figure 13. Spearman rank correlation between candidate features.
Source: COM7022 - Assignment/output/figures/fig04_feature_correlation.png

3. Model Selection

3.1 Clustering in Brief

Clustering is an **unsupervised learning** task: it partitions observations so that members of a group are more similar to one another than to members of other groups, without reference to known outcomes. It is the

appropriate paradigm here because the catalogue carries no commercial labels; the objective is to reveal structure, not to predict a target.

3.2 Candidate Techniques

Five techniques were selected because they embody different definitions of a "cluster", from classical to state-of-the-art (Table 5):

- **K-Means** (MacQueen, 1967) is centroid-based. It minimises within-cluster variance by alternately assigning points to the nearest centroid and updating the centroids:

$$J = \sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

- **Agglomerative hierarchical clustering** builds a nested tree by repeatedly merging the closest clusters. Ward linkage (Ward, 1963) merges the pair that least increases total within-cluster variance.
- **DBSCAN** (Ester et al., 1996) is density-based. It grows clusters from dense neighbourhoods defined by a radius ε and a minimum point count, and labels isolated points as noise.
- **HDBSCAN** (Campello, Moulavi and Sander, 2013; McInnes, Healy and Astels, 2017), in scikit-learn since 2023, removes ε: it builds a hierarchy over all density levels and keeps the most persistent clusters, so it can detect groups of differing density.
- **TabClustPFN** (Zhao et al., 2026) is a tabular foundation model in the TabPFN family (Hollmann et al., 2025): a transformer pre-trained on synthetic clustering problems that assigns clusters, and can predict their number, in a single pass without being fitted to the data.

Table 5. Comparison of the clustering techniques applied.

Technique	Principle	Strengths	Limitations
K-Means	Centroid-based: minimises within-cluster variance	Fast, scalable, full coverage, interpretable centroids	Requires k; assumes compact, similar-sized clusters
Hierarchical (Ward)	Agglomerative: merges clusters into a dendrogram	No k needed upfront; reveals nested structure	Irreversible merges; $O(n^2)$ memory; sensitive to sampling
DBSCAN	Density-based: grows clusters from dense regions	Arbitrary shapes; explicit noise and outlier detection	Needs density gaps; sensitive to ε and MinPts
HDBSCAN	Hierarchical density: keeps the most persistent clusters across all densities	No ε; handles varying density; membership strengths	Still needs density gaps; can leave many points as noise
TabClustPFN	Pre-trained transformer: clusters the table in context, no fitting	Predicts k itself; no tuning; seconds to run	Black box; external weights; small-data context limit

3.3 Dimensionality Reduction with PCA

Principal Component Analysis (Pearson, 1901) was applied to the standardised features, not to reduce noise (there are only three inputs) but to provide a faithful two-dimensional plane for interpreting clusters. The first two components retain **78.7%** of variance (Figure 14, Table 6). **PC1 (48.6%)** loads on sales and stock against price, measuring *commercial traction*; **PC2 (30.1%)** is dominated by price (0.91), measuring *price tier*.

Table 6. PCA loadings of the standardised features.

Feature	PC1 (48.6%)	PC2 (30.1%)	PC3 (21.3%)
log_price	-0.421	0.906	0.051
log_sold	0.636	0.335	-0.695
stock_level	0.647	0.26	0.717

Source: COM7022 - Assignment/output/tables/table06_pca_loadings.csv

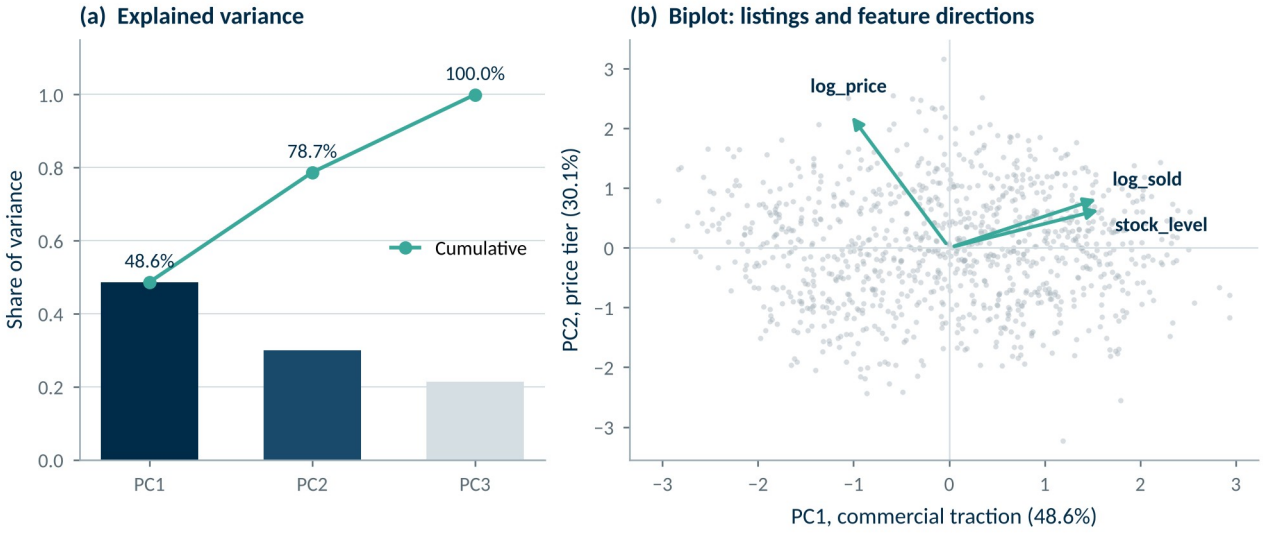


Figure 14. Explained variance (left) and loading biplot (right) of the principal components.
Source: COM7022 - Assignment/output/figures/fig05_pca_variance_biplot.png

4. Model Implementation and Evaluation

4.1 K-Means

K-Means was fitted with k-means++ initialisation, 20 restarts and a fixed seed (42) for k = 2 to 10 (Table 7, Figure 15). Inertia falls by 38.9% from k = 2 to k = 4 and flattens thereafter. The silhouette peaks at k = 2 (0.337), but a two-way split merely separates high from low volume. **k = 4** achieves the lowest Davies-Bouldin index (1.015) with a solid silhouette (0.307) and segments of 22–31% of listings. All four clusters have positive mean silhouette widths (0.29–0.32) and only 1.0% of listings carry a negative value (Figure 16).

Table 7. K-Means model scan for k = 2 to 10.

k	Inertia	Silhouette ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑	Smallest cluster
2	1943	0.337	1.211	536.7	393
3	1501	0.297	1.228	493.4	298
4	**1186.7**	**0.307**	**1.015**	**503.3**	**219**
5	974.4	0.308	1.018	513.3	156
6	862.5	0.297	1.053	489.1	141
7	766.7	0.307	1.043	478.7	84
8	691.8	0.296	1.057	469.6	82
9	640.9	0.296	1.029	452.9	59
10	594.6	0.294	1.037	442	53

Source: COM7022 - Assignment/output/tables/table07_kmeans_k_scan.csv

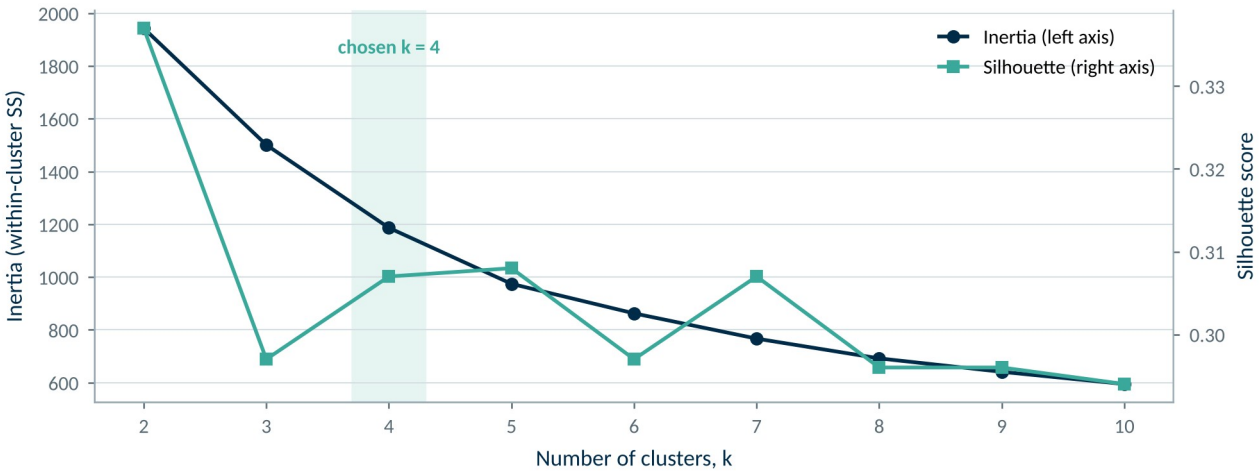


Figure 15. Elbow (inertia) and silhouette criteria across k.
Source: COM7022 - Assignment/output/figures/fig06_kmeans_elbow_silhouette.png

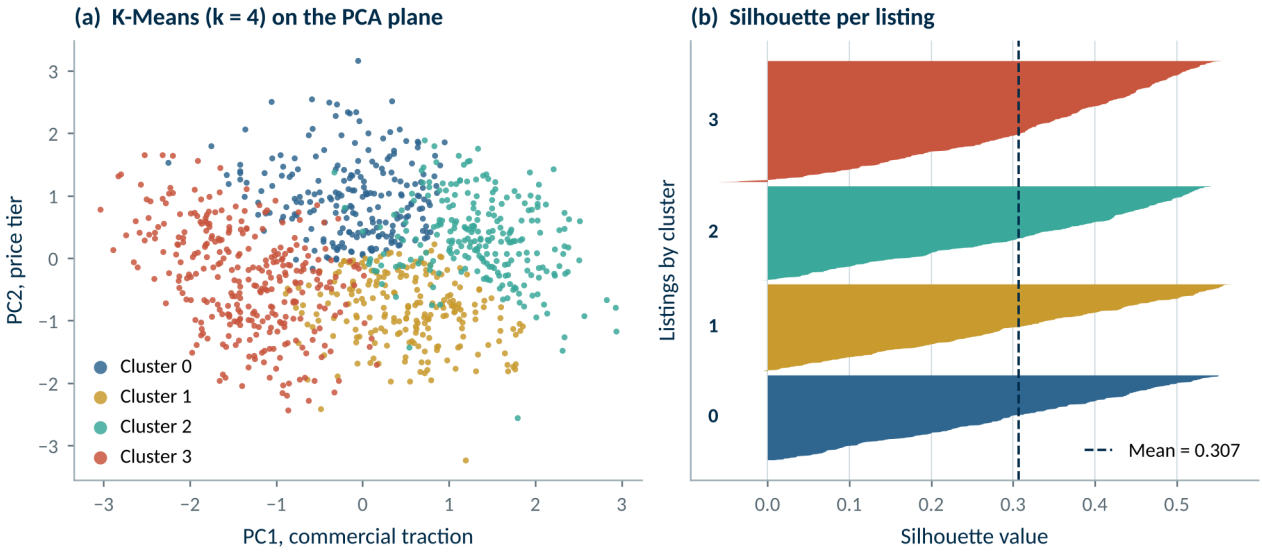


Figure 16. K-Means (k = 4) on the PCA plane and per-cluster silhouette profile.
Source: COM7022 - Assignment/output/figures/fig07_kmeans_clusters_silhouette.png

4.2 Hierarchical Clustering

Four linkage criteria were compared on identical Euclidean distances (Table 8). Single and average linkage suffer from *chaining*, absorbing 994 and 989 listings respectively into one cluster; complete linkage produces uneven groups (424/388/180/5). **Ward linkage** yields balanced, interpretable groups (358/237/235/167) and the highest silhouette among linkages (0.253). Its dendrogram separates into four branches at a merge height of roughly 21 (Figure 17), supporting a four-cluster cut that also enables a like-for-like comparison with K-Means. Agreement between the two partitions is moderate (ARI 0.41).

$$\Delta(A, B) = \frac{n_A n_B}{n_A + n_B} \|\mu_A - \mu_B\|^2$$

Table 8. Comparison of four hierarchical linkage criteria.

Linkage	Cophenetic correlation	Silhouette (k = 4)	Cluster sizes
single	0.362	0.217	994/1/1/1
complete	0.536	0.197	424/388/180/5
average	0.631	0.206	989/6/1/1
ward	0.579	0.253	358/237/235/167

Source: COM7022 - Assignment/output/tables/table08_linkage_comparison.csv

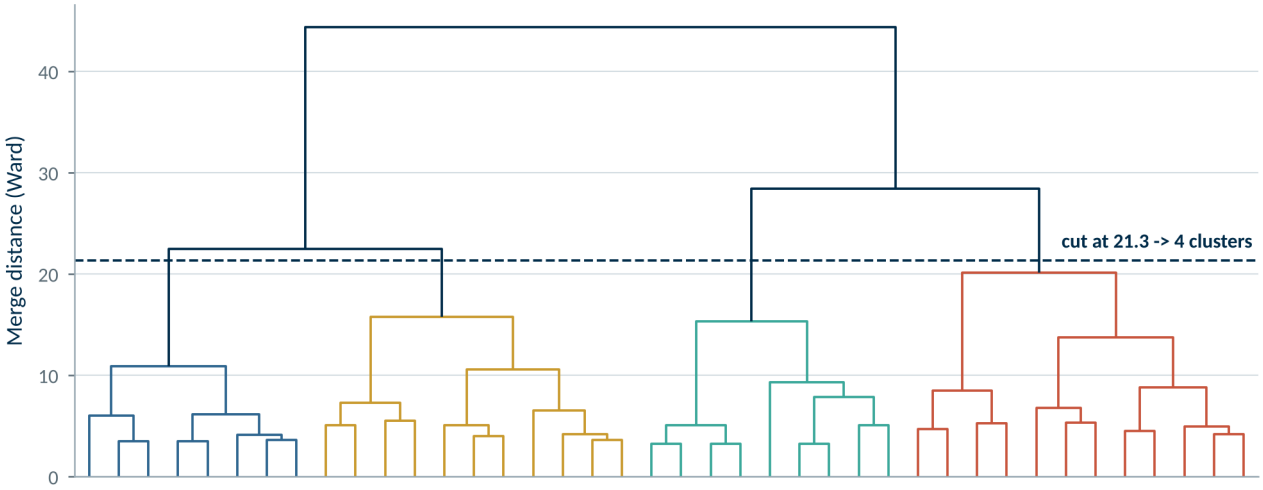


Figure 17. Ward dendrogram with the four-cluster cut.
Source: COM7022 - Assignment/output/figures/fig09_ward_dendrogram_cut.png

4.3 DBSCAN

MinPts was set to 6 (twice the dimensionality) and ε was read from the sorted 6-nearest-neighbour distance curve (Figure 18), followed by a 32-configuration grid search. The curve rises smoothly without a clear knee, a first sign that the catalogue lacks separated dense regions. Small radii (ε ≤ 0.35) reject 13–89% of listings as noise; larger radii merge almost everything. The best compromise (**ε = 0.45**) places 904 listings (90.7%) in one cluster, 4 in a second and 89 (8.9%) in noise, with a slightly negative silhouette (−0.014) (Figure 19).

This informative negative result shows that listings form a **continuum** rather than density islands. The noise set is distinctive (median price $26 vs $36, sales 163 vs 46, stock 5 vs 9), which makes DBSCAN useful as an anomaly detector.

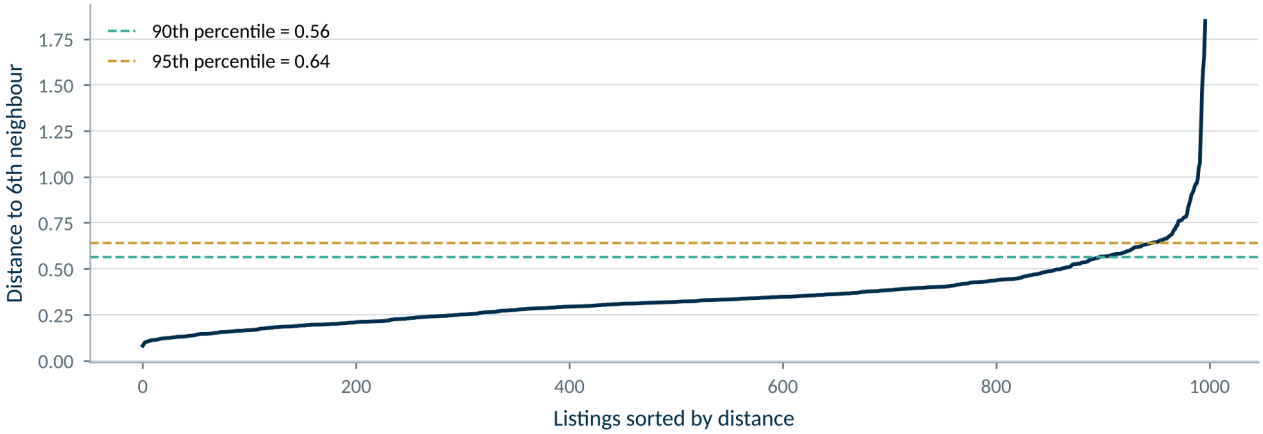


Figure 18. Sorted 6-nearest-neighbour distances used to select ε.
Source: COM7022 - Assignment/output/figures/fig10_dbscan_k_distance.png

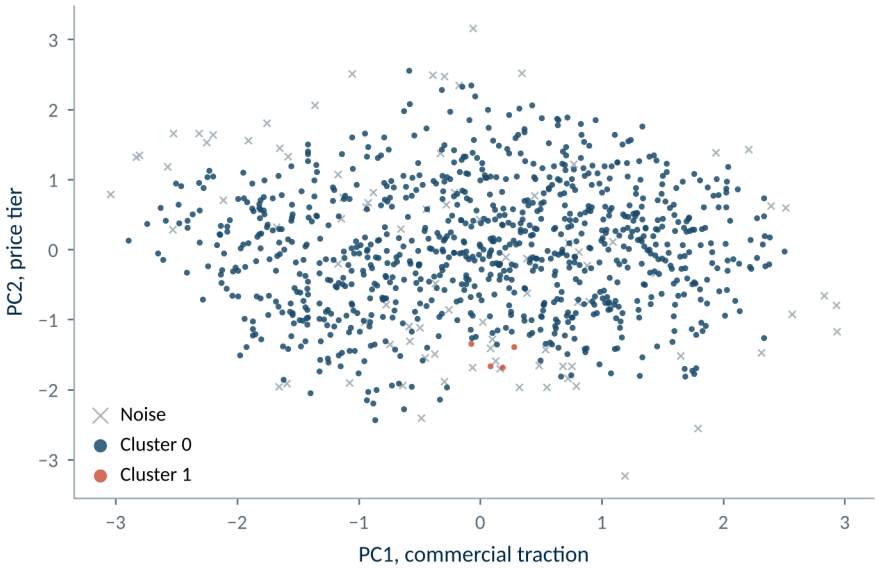


Figure 19. DBSCAN partition on the PCA plane; grey points are noise.
Source: COM7022 - Assignment/output/figures/fig11_dbscan_clusters.png

4.4 HDBSCAN

HDBSCAN directly tests whether DBSCAN failed merely because no single ε suits every region. With MinPts = 6 for comparability, a 15-configuration grid gave the same answer for every minimum cluster size from 20 to 100: **two clusters with 47.0% of listings left as noise** (Table 9, Figure 20). The larger cluster is the censoring artefact identified in Section 2.1: 89.1% of its members sit at the stock ceiling, against 41.5% of all listings. The smaller is a low-stock group (median 3 units). Jittering the integer stock scale or removing stock still leaves roughly 40–50% of listings outside any dense region. The more flexible algorithm therefore confirms the continuum and exposes a data-quality effect, but does not yield a commercial segmentation.

Table 9. HDBSCAN groups (min_cluster_size = 30, min_samples = 6) and their share of listings at the censored stock ceiling.

HDBSCAN group	Listings	Median price (US$)	Median units sold	Median stock	At stock ceiling (%)
Noise	469	38.95	51	7	13.6
Cluster 0	393	29.99	116	11	89.1
Cluster 1	135	39.99	18	3	0

Source: COM7022 - Assignment/output/tables/table16_hdbscan_profile.csv

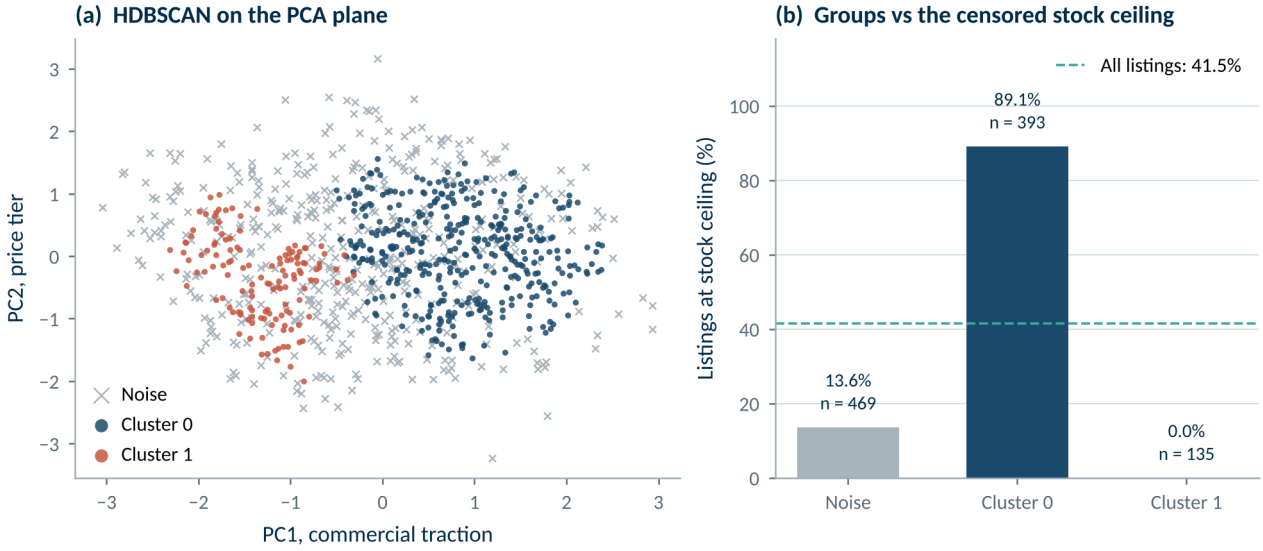


Figure 20. HDBSCAN on the PCA plane (a) and its alignment with the censored stock ceiling (b).
Source: COM7022 - Assignment/output/figures/fig18_hdbscan.png

4.5 TabClustPFN

TabClustPFN was run with the authors' pretrained checkpoint on the same standardised features, once choosing k itself and once with k fixed at 4 (Table 10, Figure 21). Inference took about three seconds on a local CPU and 44 seconds on a Colab CPU runtime. Left to decide, it predicts **two clusters**, consistently: the same k on all 30 subsamples, identical labels when the rows are shuffled, and the highest stability of any model (ARI 0.966). Its split is thin-stock listings against the rest, and 310 of the 312 listings in the K-Means low-stock segment fall in its smaller cluster. A model that never saw this data therefore confirms independently that stock position is the strongest structure in the catalogue.

Forced to four clusters it trails K-Means on every index (silhouette 0.152 against 0.307; Davies-Bouldin 1.71 against 1.02), and one group is 97% listings at the censored stock ceiling, the artefact HDBSCAN also found. The foundation model is valuable as an unbiased check on structure and on k, but it does not replace a well-prepared K-Means for a four-way business segmentation.

Table 10. TabClustPFN clusters with automatic k and with k fixed at 4.

Variant	Cluster	Listings	Median price (US$)	Median units sold	Units sold (%)	Median stock	At stock ceiling (%)
auto k = 2	0	410	39	19.5	6.9	3	0
auto k = 2	1	587	32.98	150	93.1	11	70.5
k = 4	0	150	51.99	21	0.6	8	10.7
k = 4	1	407	28.93	129	38.9	11	97.3
k = 4	2	299	37.99	16	3.2	3	0
k = 4	3	141	41	539	57.4	9	1.4

Source: COM7022 - Assignment/output/tables/table17_tabclustpfn.csv

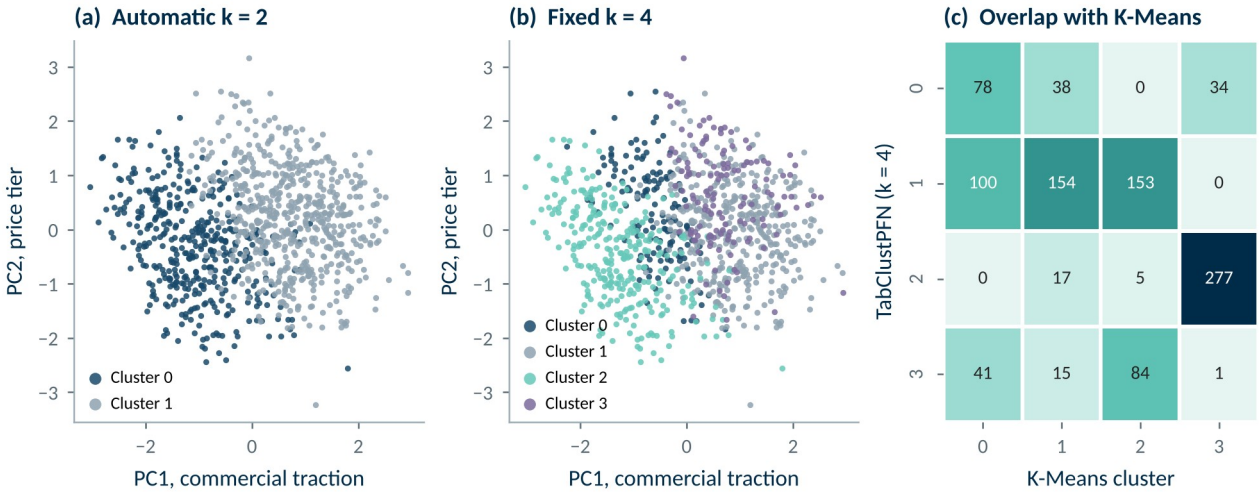


Figure 21. TabClustPFN with automatic k (a) and k = 4 (b), and the overlap of the k = 4 solution with K-Means (c).
Source: COM7022 - Assignment/output/figures/fig19_tabclustpfn.png

4.6 Evaluation Framework

Accuracy-type metrics require ground-truth labels and are therefore inapplicable to clustering. Model quality was instead triangulated across complementary measures, each answering a different question (Table 11). The silhouette coefficient (Rousseeuw, 1987) is defined for each listing as:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\, b(i)\}}$$

where a(i) is the mean distance to the listing's own cluster and b(i) to the nearest other one. Davies-Bouldin (Davies and Bouldin, 1979) and Calinski-Harabasz (Caliński and Harabasz, 1974) capture compactness against separation; the Adjusted Rand Index (Hubert and Arabie, 1985) measures agreement between partitions for the stability tests.

Table 11. Evaluation framework: complementary measures and what each can and cannot tell us.

Measure	What it captures	Better	Caveat
Silhouette	Cohesion vs separation of each listing (−1 to 1)	Higher	Favours convex clusters; ignores stability
Davies-Bouldin	Similarity of each cluster to its closest rival	Lower	Assumes centroid-shaped clusters
Calinski-Harabasz	Between- to within-cluster dispersion ratio	Higher	Inflated for convex clusters; weak for DBSCAN
Bootstrap stability (ARI)	Agreement of labels over 30 resamples of 80%	Higher (max 1)	Measures reproducibility, not quality
Sensitivity (ARI)	Agreement after changing seed, scaling or outliers	Higher (max 1)	Tests robustness to analyst choices
Coverage and balance	Share of noise; smallest and largest cluster	Low noise, balanced	Business usability rather than geometry

4.7 Comparative Results

Table 12. Comparative performance of the final models (density models scored on clustered listings only; TCP = TabClustPFN).

Model	Clusters	Noise %	Silhouette ↑	DBI ↓	CH ↑	Stability ARI ↑	Smallest %	Largest %
K-Means	**4**	**0**	**0.307**	**1.015**	**503.3**	**0.932 ± 0.025**	**22**	**31.3**
Ward	4	0	0.253	1.257	402.9	0.481 ± 0.084	16.8	35.9
DBSCAN	2	8.9	-0.014	1.172	3.5	0.749 ± 0.063	0.4	90.7
HDBSCAN	2	47	0.48	0.785	476	0.705 ± 0.147	13.5	39.4
TCP (auto k)	2	0	0.326	1.261	497.6	0.966 ± 0.024	41.1	58.9
TCP (k = 4)	4	0	0.152	1.71	278.7	0.933 ± 0.036	14.1	40.8

Source: COM7022 - Assignment/output/tables/table11_model_comparison.csv

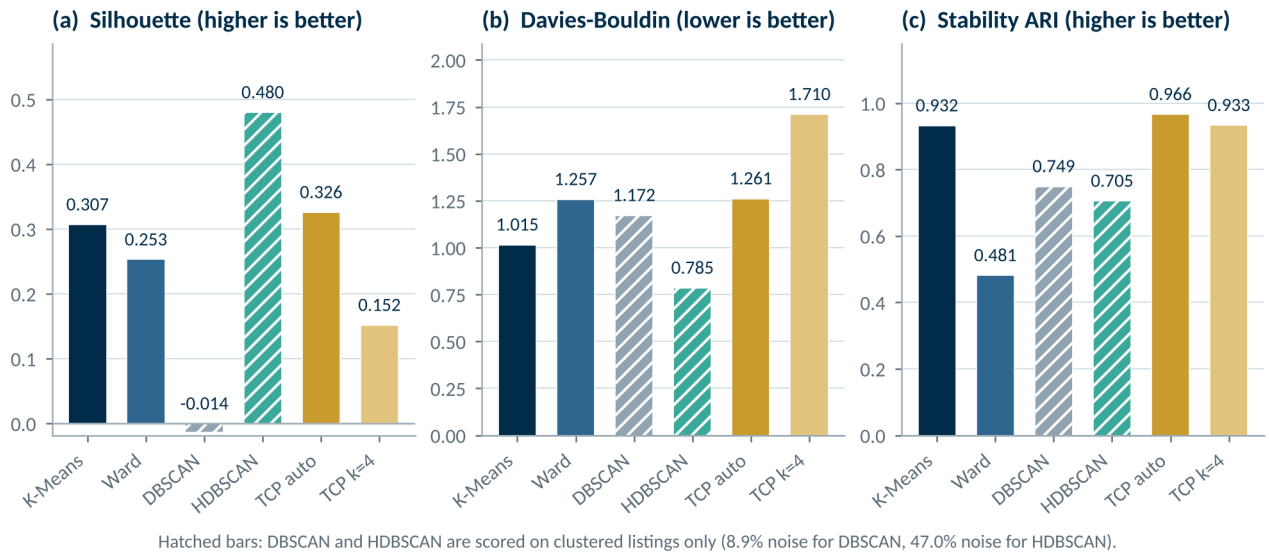


Figure 22. Internal validity and stability by model; hatched bars are scored on clustered listings only.
Source: COM7022 - Assignment/output/figures/fig12_model_comparison.png

Among models that assign every listing, K-Means leads on **every internal index**: silhouette 0.307 versus 0.253 for Ward, Davies-Bouldin 1.015 versus 1.257, Calinski-Harabasz 503.3 versus 402.9. TabClustPFN's automatic two-cluster solution scores a similar silhouette (0.326) to K-Means at the same k (0.337), but at k = 4 it is clearly weaker. HDBSCAN's higher silhouette (0.480) is not comparable, because it is computed on the 53% of listings it retains; scoring only the easiest points inflates every index. Stability is decisive: across 30 resamples of 80% of the data, K-Means reproduces its partition almost exactly (**ARI 0.932 ± 0.025**), whereas Ward does not (0.481 ± 0.084) because early merges in a hierarchy are irreversible and small sample changes propagate up the tree. TabClustPFN is equally stable (0.966 automatic, 0.933 at k = 4), so stability alone does not separate the two; segment quality at k = 4 does.

Sensitivity tests confirm robustness (Table 13): changing the seed (mean ARI 0.984) or removing the top 1% of sellers (0.937) barely alters the solution, so the log transform neutralised outlier leverage. Only min-max scaling (0.492) materially changes it, justifying standardisation.

Table 13. Robustness of the K-Means solution to analytical choices.

Sensitivity test (K-Means, k = 4)	ARI
K-Means across 10 seeds (mean ARI)	0.984
z-score vs min-max scaling (ARI)	0.492
with vs without top 1% sellers (ARI)	0.937

Source: COM7022 - Assignment/output/tables/table12_sensitivity_checks.csv

> **Model Selection Decision: K-Means (k = 4)** is selected as the production segmentation engine: best internal validity among full-coverage models, near-perfect stability and full catalogue coverage. Ward is retained as a structural cross-check, DBSCAN as an anomaly flag, HDBSCAN as a data-quality probe and TabClustPFN as an independent check on the number of segments.

5. Cluster Characteristics

Mapping centroids back to business units reveals four distinct commercial segments (Table 14, Figure 23, Figure 24):

- **Cluster 2 — Best-Sellers (24.3% of listings).** The revenue engine: **92.3% of all units sold** (710,124 of 769,317) at an accessible median $29.94, with full stock. Mostly Eau de Toilette from lines such as Armaf and Ralph Lauren.
- **Cluster 0 — Premium Designer (22.0%).** The highest prices (median $68.99) with steady but slow demand (3.7% of volume); Eau de Parfum from Giorgio Armani, Paco Rabanne and Yves Saint Laurent.
- **Cluster 1 — Budget Commodities (22.5%).** The cheapest offers (median $18.99) yet only 1.8% of volume, with the most unbranded items: low price alone does not create demand.
- **Cluster 3 — Low-Stock Tail (31.3%).** Median stock of 3 units, 14.1% at "Last One" and minimal sales (2.2% of volume): an imminent stock-out risk.

Table 14. Business profile of the four K-Means segments.

Measure	C2 Best-Sellers	C0 Premium	C1 Budget	C3 Low-Stock
Listings	242	219	224	312
Share of listings (%)	24.3	22	22.5	31.3
Median price (US$)	29.94	68.99	18.99	39.99
Median units sold	952	43	30	16
Share of units sold (%)	92.3	3.7	1.8	2.2
Median stock level	11	11	11	3
Listings at 'Last One' (%)	1.2	0	0	14.1
Ships from US (%)	99.6	84.9	92	88.5
Dominant product type	Eau de Toilette (67.8%)	Eau de Parfum (45.2%)	Eau de Toilette (37.9%)	Eau de Toilette (39.1%)

Source: COM7022 - Assignment/output/tables/table13_cluster_profiles.csv



Figure 23. Standardised feature means per cluster.
Source: COM7022 - Assignment/output/figures/fig13_cluster_profiles.png

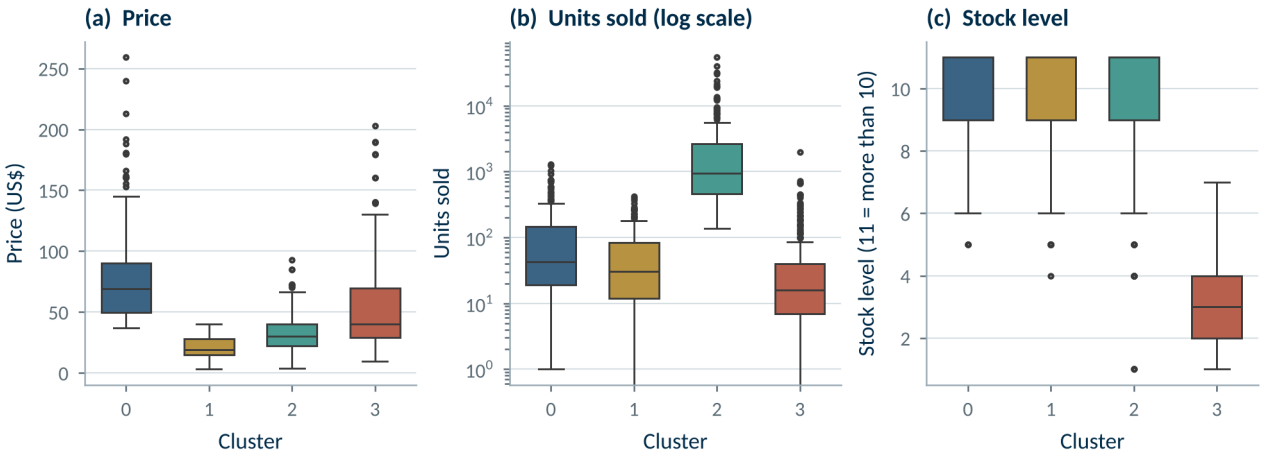


Figure 24. Price, sales and stock distributions within each cluster.
Source: COM7022 - Assignment/output/figures/fig14_cluster_distributions.png

TASK 2

Critical Report for Executive Stakeholders

947 words of body text

6. Executive Summary

Machine learning reveals that our 997 fragrance listings fall into **four natural commercial segments**. One quarter of listings — the best-sellers — generate **92% of all units sold**, while almost a third of the catalogue is close to running out of stock. The recommended model, **K-Means with four segments**, produced better segments than four alternatives, including a 2026 AI foundation model, is among the most stable, and is simple enough to run automatically on new listings. Each segment calls for a distinct action: protect, grow, reposition or audit.

CORE REVENUE ENGINE	INVENTORY RISK	MODEL STABILITY	PCA VARIANCE
92.3%	**31.3%**	**0.932**	**78.7%**
of units sold come from 24.3% of listings (Cluster 2)	of listings sit in the low-stock tail (Cluster 3)	K-Means bootstrap ARI over 30 resamples	retained by the first two components

7. Critical Evaluation

7.1 Suitability of the Methods and Design Decisions

Unsupervised clustering is the right tool because the company holds no labelled outcomes; the aim is discovery rather than prediction. Three design decisions shaped the outcome. The log transformation stopped a handful of viral listings from dictating the segmentation. The choice of price, sales and stock answers the three questions managers actually ask — *what is it worth, does it sell, will it run out?* Excluding brand and country from the distance calculation prevented segments from degenerating into brand lists. Each decision was tested rather than assumed: the result survives changes of seed and the removal of top sellers.

7.2 Comparative Performance: Strengths and Limitations

K-Means was the strongest model on every measure. Its limitation is that it assumes compact, similarly sized groups and requires the number of segments to be chosen; on a continuous catalogue its boundaries are therefore soft, as the moderate silhouette (0.31) honestly reflects. **Ward hierarchical clustering** offers an intuitive tree of nested groups but proved unstable: its segments changed substantially whenever the sample changed, and it cannot assign new listings without rebuilding the tree. **DBSCAN** excels at irregular shapes and outlier detection, but placed 91% of listings in one group. **HDBSCAN**, the newest technique tested, removes DBSCAN's fragile radius setting, yet still left almost half the catalogue unassigned, and its densest cluster reflected how the platform records stock rather than how listings sell. **TabClustPFN**, a pre-trained foundation model, needs no tuning and is very stable, and its automatic choice of two clusters is a useful sanity check; but it is a black box that depends on externally hosted weights, and when asked for four segments it produced weaker, less commercially meaningful groups than K-Means. Newer is therefore not automatically better: on a small, well-understood table, careful preparation and a transparent model win. These weaknesses are

properties of the data as much as of the algorithms, which is itself a valuable insight into how the marketplace is structured.

7.3 Why K-Means Is the Right Choice for E-Commerce

Technical scores are only half the decision. An e-commerce platform needs a model that covers every listing, can be refreshed frequently, scores new listings instantly, scales to millions of offers and can be explained to managers (Table 15). K-Means satisfies all five requirements: each new listing is assigned to its nearest centroid in microseconds, and each centroid reads as a typical price, sales level and stock position. A simple sales ranking would not suffice: it shows who sells most, but not who is premium, cheap yet stagnant, or about to run out.

Table 15. Operational suitability of each model for an e-commerce platform.

Requirement	K-Means	Ward	DBSCAN	HDBSCAN	TabClustPFN
Covers every listing	Yes · 0% noise	Yes · 0% noise	No · 8.9% noise	No · 47.0% noise	Yes · 0% noise
Scores new listings instantly	Yes · nearest centroid	No · rebuild tree	Partly	Partly	Partly · re-run (~3 s)
Stable between refreshes	Yes · ARI 0.93	No · ARI 0.48	Partly · 0.75	Partly · 0.71	Yes · ARI 0.93–0.97
Scales to millions of listings	Yes · linear	No · quadratic	Partly	Partly	No · context limit
Explainable to managers	Yes · four profiles	Partly	No · one group	No · half noise	Partly · black box
Recommended role	Production model	Cross-check	Anomaly flag	Data-quality probe	Check on k

8. Recommendations for Business Managers

The four segments translate into targeted, measurable interventions (Figure 25):

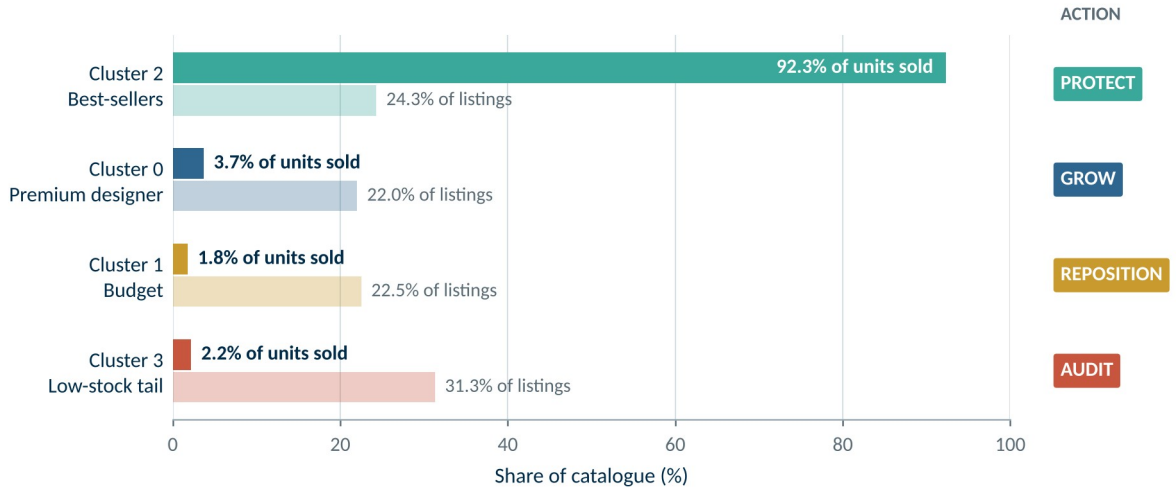


Figure 25. Share of listings versus share of units sold per segment, with the recommended action.
Source: COM7022 - Assignment/output/figures/fig17_segment_actions.png

PROTECT · CLUSTER 2

Secure the revenue engine

Evidence: 92.3% of units sold from 24.3% of listings.

Action: Automated restock alerts below 10 units, priority search placement and fulfilment SLAs for these sellers.

GROW · CLUSTER 0

Monetise the premium tier

Evidence: Highest prices (median $68.99) with steady, slow demand.

Action: Bundle with best-sellers on product pages and invest in premium content rather than discounting.

REPOSITION · CLUSTER 1

Stop subsidising weak demand

Evidence: Cheapest offers (median $18.99) yet only 1.8% of volume.

Action: End paid promotion of unbranded items and reallocate budget to proven brands.

AUDIT · CLUSTER 3

Clean up the low-stock tail

Evidence: 31.3% of listings, median stock of 3, 14.1% at 'Last One'.

Action: 30-day merchant restock notices; delist listings that remain empty and unsold after 60 days.

Implementation should begin with the best-sellers and the low-stock tail, which protect the most revenue for the least effort. Success should be tracked through stock-outs in Cluster 2, the proportion of Cluster 3 listings restocked or delisted within 30 days, and sales per advertising dollar in Cluster 1. The segmentation should be refreshed monthly, with new listings scored on arrival and DBSCAN noise points reviewed as potential anomalies.

9. Limitations

The platform caps visible stock at ten units, so genuine depth above that level is unknown. The data is a single snapshot of one product family, so segment boundaries may differ in other categories or seasons. Clusters describe association, not causation: a premium price does not itself make a listing slow. Finally, because listings form a continuum, those near a boundary could reasonably belong to either neighbouring segment; the segments should guide prioritisation rather than replace commercial judgement.

10. Conclusion and Future Work

This project demonstrates that a carefully prepared, well-validated K-Means model can turn an unlabelled and messy catalogue into four clear, actionable segments, and that the commercial value of the catalogue is extraordinarily concentrated. The most important lesson is that **data preparation mattered more than algorithm choice**, even against a state-of-the-art foundation model: recovering values from listing text and correcting the hidden stock ceiling changed the outcome more than any hyperparameter. Equally, the density methods were not wasted effort: DBSCAN and HDBSCAN together provided the clearest evidence that the marketplace is a continuum, and HDBSCAN exposed how stock censoring distorts the data. TabClustPFN added an independent, pre-trained view that agreed on the dominant stock split.

Future work would enrich the feature space with revenue, margin and true stock counts (removing the censoring that misled two of the models), incorporate text embeddings of product descriptions, and track segment migration over time to detect listings that are rising or declining. Validating whether segment membership predicts next-quarter sales would convert the descriptive model into a forward-looking decision tool, and a lightweight dashboard would place each listing's segment directly in the hands of category managers.

References

Campello, R.J.G.B., Moulavi, D. and Sander, J. (2013) 'Density-based clustering based on hierarchical density estimates', in *Advances in Knowledge Discovery and Data Mining (PAKDD 2013)*. Berlin: Springer, pp. 160–172.

Caliński, T. and Harabasz, J. (1974) 'A dendrite method for cluster analysis', *Communications in Statistics*, 3(1), pp. 1–27.

Davies, D.L. and Bouldin, D.W. (1979) 'A cluster separation measure', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2), pp. 224–227.

Ester, M., Kriegel, H.-P., Sander, J. and Xu, X. (1996) 'A density-based algorithm for discovering clusters in large spatial databases with noise', in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*. Portland: AAAI Press, pp. 226–231.

Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., Körfer, M., Hoo, S.B., Schirrmeister, R.T. and Hutter, F. (2025) 'Accurate predictions on small data with a tabular foundation model', *Nature*, 637, pp. 319–326.

Hubert, L. and Arabie, P. (1985) 'Comparing partitions', *Journal of Classification*, 2(1), pp. 193–218.

MacQueen, J. (1967) 'Some methods for classification and analysis of multivariate observations', in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. Berkeley: University of California Press, pp. 281–297.

McInnes, L., Healy, J. and Astels, S. (2017) 'hdbscan: Hierarchical density based clustering', *Journal of Open Source Software*, 2(11), p. 205.

McKinney, W. (2010) 'Data structures for statistical computing in Python', in *Proceedings of the 9th Python in Science Conference*, pp. 56–61.

Müller, A.C. and Guido, S. (2016) *Introduction to Machine Learning with Python*. Sebastopol: O'Reilly Media.

Pearson, K. (1901) 'On lines and planes of closest fit to systems of points in space', *Philosophical Magazine*, 2(11), pp. 559–572.

Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Rousseeuw, P.J. (1987) 'Silhouettes: a graphical aid to the interpretation and validation of cluster analysis', *Journal of Computational and Applied Mathematics*, 20, pp. 53–65.

Ward, J.H. (1963) 'Hierarchical grouping to optimize an objective function', *Journal of the American Statistical Association*, 58(301), pp. 236–244.

Zhao, T., Wang, G., Tan, Y.S. and Zhang, Q. (2026) 'TabClustPFN: a prior-fitted network for tabular data clustering', *arXiv preprint* arXiv:2601.21656.

Appendix A: Reproducible Code

The complete Task 1 implementation is a single Jupyter notebook that executes end to end in about 30 seconds on Google Colab or locally, with fixed random seeds (42). All materials — notebook, dataset, figures, tables and code screenshots — are hosted in the project repository: https://github.com/kvmmn/_COM7022_MachineLearning. TabClustPFN is run from the authors' public code and checkpoint (CC BY-NC 4.0 licence, used for non-commercial coursework); the notebook downloads both and, because R's `rpy2` is only needed to generate the model's pre-training data, supplies a placeholder module that raises an error if ever called. The notebook was executed on Google Colab (scikit-learn 1.6.1, no GPU needed) as well as locally (1.9.1), and every reported figure reproduced; the only difference is in the third decimal of one standard deviation (HDBSCAN stability, 0.149 on Colab against 0.147 locally). The full notebook code follows, one screenshot per cell in execution order.

A · 0. Set-up

COM7022_Clustering_25199053.ipynb | **code cell 1**

```
# Standard libraries
import os
import re
import json
import warnings
from pathlib import Path

# Data handling and plotting
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
import scipy
import sklearn

# Machine learning tools
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN, HDBSCAN
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import (silhouette_score, silhouette_samples, davies_bouldin_score,
                             calinski_harabasz_score, adjusted_rand_score)
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster, cophenet, set_link_color_palette
from scipy.spatial.distance import pdist
from scipy.stats import skew

warnings.filterwarnings("ignore", category=FutureWarning)

# One seed for every random step, so a re-run gives the same numbers
SEED = 42
np.random.seed(SEED)

print("Python libraries used in this run:")
print("pandas", pd.__version__, "| numpy", np.__version__, "| scikit-learn", sklearn.__version__,
      "| scipy", scipy.__version__, "| matplotlib", matplotlib.__version__, "| seaborn", sns.__version__)
```

Code A1. Notebook code cell 1

Source: COM7022 - Assignment/output/screenshots/appendix/cell01.png

COM7022_Clustering_25199053.ipynb | **code cell 2**

```
# Arden University palette (Midnight / Tiffany), the same visual system used in my earlier modules
MIDNIGHT = "#002B49"
TIFFANY = "#66C9BA"
TIFFANY_DEEP = "#3AA89A"
MIDNIGHT_SOFT = "#1A4A6B"
TEAL = "#0E7C8B"
MIST = "#E6F5F2"
PAPER = "#F7FAFB"
MUTE = "#5A6B75"
GRID = "#D5DEE3"
NOISE_GREY = "#A7B4BC"

# One fixed colour per K-Means segment, used in every chart and in the report (index = cluster label):
# 0 Premium = steel blue, 1 Budget = ochre, 2 Best-sellers = tiffany, 3 Low-stock = terracotta.
CLUSTER_COLOURS = ["#2F6690", "#C99A2E", TIFFANY_DEEP, "#C8553D", MIDNIGHT]
MODEL_COLOURS = {"K-Means": MIDNIGHT, "Hierarchical (Ward)": "#2F6690", "DBSCAN": "#8FA3B1",
                 "HDBSCAN": TIFFANY_DEEP, "TabClustPFN (auto k)": "#C99A2E", "TabClustPFN (k = 4)": "#E2C37E"}
ARDEN_CMAP = matplotlib.colors.LinearSegmentedColormap.from_list(
    "arden", [MIST, TIFFANY, TIFFANY_DEEP, MIDNIGHT_SOFT, MIDNIGHT])
ARDEN_DIVERGING = matplotlib.colors.LinearSegmentedColormap.from_list(
    "arden_div", ["#C8553D", "#F3D9D2", "#FFFFFF", "#CFE3EE", MIDNIGHT_SOFT])

# Print-ready figure system. Every chart is drawn at its final printed size (the report's text width is
# 6.6 in = 16.8 cm), so labels are the same 8.5 pt in every figure and nothing is rescaled in the report.
TEXT_WIDTH = 6.6
NARROW = 4.6
plt.rcParams.update({
    "font.family": "sans-serif",
    "font.sans-serif": ["Carlito", "Calibri", "Arial", "DejaVu Sans"],
    "font.size": 8.5,
    "axes.titlesize": 9,
    "axes.titleweight": "bold",
    "axes.titlelocation": "left",
    "axes.titlepad": 6,
    "axes.labelsize": 8.5,
    "xtick.labelsize": 7.5,
    "ytick.labelsize": 7.5,
    "legend.fontsize": 7.5,
    "figure.facecolor": "white",
    "axes.facecolor": "white",
    "axes.edgecolor": "#9AAAB4",
    "axes.linewidth": 0.6,
    "axes.labelcolor": MIDNIGHT,
    "axes.titlecolor": MIDNIGHT,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "axes.grid.axis": "y",
    "axes.axisbelow": True,
    "grid.color": GRID,
    "grid.linewidth": 0.5,
    "xtick.color": MUTE,
    "ytick.color": MUTE,
    "xtick.major.size": 2.5,
    "ytick.major.size": 2.5,
    "xtick.major.width": 0.6,
    "ytick.major.width": 0.6,
    "lines.linewidth": 1.4,
    "lines.markersize": 4,
    "patch.linewidth": 0.4,
    "legend.frameon": False,
    "legend.handlelength": 1.4,
    "figure.dpi": 100,
    "savefig.dpi": 300,
    "savefig.bbox": "standard",
    "savefig.facecolor": "white",
    "figure.constrained_layout.use": True,
    "figure.constrained_layout.h_pad": 0.04,
    "figure.constrained_layout.w_pad": 0.04,
})


def panel(ax, letter, title):
    # Panel titles are lettered so the report caption can refer to (a), (b), (c)
    ax.set_title("(" + letter + ")  " + title)


# Output folders for the figures and tables used in the report
OUTPUT_DIR = Path("output")
FIG_DIR = OUTPUT_DIR / "figures"
TABLE_DIR = OUTPUT_DIR / "tables"
FIG_DIR.mkdir(parents=True, exist_ok=True)
TABLE_DIR.mkdir(parents=True, exist_ok=True)


def save_figure(fig, filename):
    # Save every chart with the same settings so the report pack is consistent
    fig.savefig(FIG_DIR / filename)
    print("Saved:", FIG_DIR / filename)


def save_table(table, filename):
    table.to_csv(TABLE_DIR / filename)
    print("Saved:", TABLE_DIR / filename)
```

Code A2. Notebook code cell 2

Source: COM7022 - Assignment/output/screenshots/appendix/cell02.png

A · 1. Data import

COM7022_Clustering_25199053.ipynb | **code cell 3**

```
DATA_FILE = "Sales-[5444].csv"
candidate_paths = [Path(DATA_FILE), Path("..") / DATA_FILE, Path("/content") / DATA_FILE]

data_path = None
for path in candidate_paths:
    if path.exists():
        data_path = path
        break

if data_path is None:
    try:
        from google.colab import files  # only available on Colab
        print("CSV not found. Please upload", DATA_FILE)
        uploaded = files.upload()
        data_path = Path(list(uploaded.keys())[0])
    except ImportError:
        raise FileNotFoundError("Place " + DATA_FILE + " next to this notebook.")

raw = pd.read_csv(data_path)
print("Loaded:", data_path)
print("Shape:", raw.shape[0], "rows x", raw.shape[1], "columns")
raw.head()
```

Code A3. Notebook code cell 3

Source: COM7022 - Assignment/output/screenshots/appendix/cell03.png

COM7022_Clustering_25199053.ipynb | **code cell 4**

```
# Column types and non-null counts
raw.info()
```

Code A4. Notebook code cell 4

Source: COM7022 - Assignment/output/screenshots/appendix/cell04.png

A · 2. Descriptive statistics and data quality

COM7022_Clustering_25199053.ipynb | **code cell 5**

```
# Missing values per column, as counts and percentages
missing = pd.DataFrame({
    "missing_count": raw.isna().sum(),
    "missing_pct": (raw.isna().mean() * 100).round(1),
})
missing = missing.sort_values("missing_count", ascending=False)
print("Full-row duplicates:", raw.duplicated().sum())
print("Duplicate listing descriptions:", raw["details"].duplicated().sum())
save_table(missing, "table01_missing_values.csv")
missing
```

Code A5. Notebook code cell 5

Source: COM7022 - Assignment/output/screenshots/appendix/cell05.png

COM7022_Clustering_25199053.ipynb | **code cell 6**

```
# Figure 1: where the gaps are
data = missing["missing_count"][::-1]
colours = [GRID if v == 0 else MIDNIGHT_SOFT for v in data.values]
colours[-1] = TIFFANY_DEEP          # the biggest gap drives a pre-processing decision
fig, ax = plt.subplots(figsize=(NARROW, 2.4))
bars = ax.barh(data.index, data.values, color=colours, height=0.62)
for bar, value in zip(bars, data.values):
    ax.text(bar.get_width() + 1.5, bar.get_y() + bar.get_height() / 2, str(value), va="center",
            color=MIDNIGHT, fontsize=7.5)
ax.set_xlim(0, data.max() * 1.12)
ax.set_xlabel("Missing values (n = 1,000 listings)")
ax.grid(axis="y", visible=False)
ax.grid(axis="x", visible=True)
save_figure(fig, "fig01_missing_values.png")
plt.show()
```

Code A6. Notebook code cell 6

Source: COM7022 - Assignment/output/screenshots/appendix/cell06.png

COM7022_Clustering_25199053.ipynb | **code cell 7**

```
# Descriptive statistics for the numeric columns, including skewness
numeric_cols = ["price", "available", "sold"]
desc = raw[numeric_cols].describe().T
desc["median"] = raw[numeric_cols].median()
desc["skewness"] = [skew(raw[c].dropna()) for c in numeric_cols]
desc = desc[["count", "mean", "median", "std", "min", "25%", "75%", "max", "skewness"]].round(2)
save_table(desc, "table02_descriptive_statistics_raw.csv")
desc
```

Code A7. Notebook code cell 7

Source: COM7022 - Assignment/output/screenshots/appendix/cell07.png

COM7022_Clustering_25199053.ipynb | **code cell 8**

```
# Text and category columns: how many distinct values, and the most common ones
for col in ["Item_Name", "Catatory", "itemLocation"]:
    print(col, "->", raw[col].nunique(), "distinct values")
print()
print("Category labels (top 15 of", raw["Catatory"].nunique(), "):")
print(raw["Catatory"].value_counts().head(15))
```

Code A8. Notebook code cell 8

Source: COM7022 - Assignment/output/screenshots/appendix/cell08.png

COM7022_Clustering_25199053.ipynb | **code cell 9**

```
# The 'text' column holds the availability and sales wording shown on the listing.
# Replacing digits with N shows the small number of patterns behind 1,000 rows.
patterns = raw["text"].fillna("<missing>").str.split("/").str[0].str.strip()
patterns = patterns.str.replace(r"\d[\d,]*", "N", regex=True)
print(patterns.value_counts())
print()
print("Currencies in priceWithCurrency:")
print(raw["priceWithCurrency"].str.extract(r"^([A-Z]+ ?\$)")[0].value_counts())
```

Code A9. Notebook code cell 9

Source: COM7022 - Assignment/output/screenshots/appendix/cell09.png

COM7022_Clustering_25199053.ipynb | **code cell 10**

```
# Figure 2: raw distributions of price and units sold
fig, axes = plt.subplots(1, 2, figsize=(TEXT_WIDTH, 2.4))
for ax, col, colour, letter, title, xlabel in [
        (axes[0], "price", MIDNIGHT_SOFT, "a", "Price (raw)", "Price (US$)"),
        (axes[1], "sold", TIFFANY_DEEP, "b", "Units sold (raw)", "Units sold")]:
    values = raw[col].dropna()
    ax.hist(values, bins=40, color=colour, edgecolor="white", linewidth=0.3)
    ax.axvline(values.median(), color=MIDNIGHT, linestyle="--", linewidth=0.9)
    ax.text(0.97, 0.92, "median " + format(values.median(), ",.1f") + "\nskewness " + format(skew(values), ".2f"),
            transform=ax.transAxes, ha="right", va="top", fontsize=7.5, color=MIDNIGHT)
    panel(ax, letter, title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel("Listings")
save_figure(fig, "fig02_raw_distributions.png")
plt.show()
```

Code A10. Notebook code cell 10

Source: COM7022 - Assignment/output/screenshots/appendix/cell10.png

A · 3. Pre-processing

COM7022_Clustering_25199053.ipynb | **code cell 11**

```
df = raw.copy()
cleaning_log = []


def log_step(step, rows_affected, note):
    cleaning_log.append({"step": step, "rows_affected": int(rows_affected), "note": note})


# 3.1 Price: recover missing prices from priceWithCurrency (same listing, same number)
price_from_text = df["priceWithCurrency"].str.extract(r"\$([\d,]+\.?\d*)")[0]
price_from_text = price_from_text.str.replace(",", "").astype(float)
recovered = df["price"].isna() & price_from_text.notna()
df.loc[recovered, "price"] = price_from_text[recovered]
log_step("Recover price from priceWithCurrency", recovered.sum(), "Exact same value, only stored in text form")

# Check: where both exist, do they agree?
both = df["price"].notna() & price_from_text.notna()
print("Price disagreements where both exist:", (abs(df.loc[both, "price"] - price_from_text[both]) > 0.01).sum())

# 3.2 Currency: convert the three Canadian-dollar prices to USD
# Approximate May 2024 rate (1 CAD ~ 0.73 USD). Only 3 rows, so the exact rate barely matters.
CAD_TO_USD = 0.73
is_cad = df["priceWithCurrency"].fillna("").str.startswith("C")
df.loc[is_cad, "price"] = (df.loc[is_cad, "price"] * CAD_TO_USD).round(2)
log_step("Convert CAD prices to USD", is_cad.sum(), "Fixed rate 0.73, documented approximation")
```

Code A11. Notebook code cell 11

Source: COM7022 - Assignment/output/screenshots/appendix/cell11.png

COM7022_Clustering_25199053.ipynb | **code cell 12**

```
# 3.3 Units sold: recover from the 'text' column ("... / 116 sold")
sold_from_text = df["text"].str.extract(r"([\d,]+) sold")[0].str.replace(",", "").astype(float)
recovered = df["sold"].isna() & sold_from_text.notna()
df.loc[recovered, "sold"] = sold_from_text[recovered]
log_step("Recover sold from text", recovered.sum(), "Exact value from the listing wording")

# Check the two sources agree everywhere else
both = df["sold"].notna() & sold_from_text.notna()
print("Sold disagreements where both exist:", (df.loc[both, "sold"] != sold_from_text[both]).sum())

# Listings whose text shows stock but no 'sold' part have not sold yet, so sold = 0
no_sales_yet = df["sold"].isna() & df["text"].notna() & ~df["text"].str.contains("sold", na=False)
df.loc[no_sales_yet, "sold"] = 0
log_step("Set sold = 0 where text shows stock but no sales", no_sales_yet.sum(), "Listing wording omits 'sold' when nothing has sold")

# Rows with no text and no sold value carry no sales or stock information at all
no_info = df["sold"].isna() & df["text"].isna()
print(df.loc[no_info, ["Item_Name", "price", "available", "text", "sold"]])
df = df[~no_info].copy()
log_step("Drop rows with no sales or stock information", no_info.sum(), "Cannot be placed on the sales-performance axis")
```

Code A12. Notebook code cell 12

Source: COM7022 - Assignment/output/screenshots/appendix/cell12.png

COM7022_Clustering_25199053.ipynb | **code cell 13**

```
# 3.4 Stock level: rebuild it from the listing wording so every row has a value.
# The marketplace hides exact numbers above 10 ("More than 10 available"), so I cap all
# stock values at 11, meaning "more than 10". This keeps the measure honest and comparable.
text = df["text"].fillna("")
stock = text.str.extract(r"^(\d+) available")[0].astype(float)
stock[text.str.startswith("More than")] = 11
stock[text.str.startswith("Last One")] = 1
capped_above_10 = (stock > 11).sum()
stock = stock.clip(upper=11)

# "Limited quantity available" and lot listings give no number: impute the median, but keep a flag
stock_imputed = stock.isna()
df["stock_imputed"] = stock_imputed.astype(int)
stock = stock.fillna(stock.median())
df["stock_level"] = stock

log_step("Stock: 'More than 10' coded as 11", text.str.startswith("More than").sum(), "Censored value, not exact")
log_step("Stock: explicit values above 10 capped at 11", capped_above_10, "Same scale as the censored rows")
log_step("Stock: 'Last One' coded as 1", text.str.startswith("Last One").sum(), "Wording means one unit left")
log_step("Stock: median imputation", stock_imputed.sum(), "Limited quantity / lot wording; flagged in stock_imputed")
print(df["stock_level"].value_counts().sort_index())
```

Code A13. Notebook code cell 13

Source: COM7022 - Assignment/output/screenshots/appendix/cell13.png

COM7022_Clustering_25199053.ipynb | **code cell 14**

```
# 3.5 Product type: collapse the messy category labels into six groups


def group_category(label):
    if pd.isna(label):
        return "Other"
    s = str(label).strip().lower()
    if s in ["", "/", "y", "does not apply", "unknown", "various", "assorted"]:
        return "Other"
    if "," in s or "/" in s:
        return "Other"                          # listings that name several types at once
    if "toil" in s or s == "edt":
        return "Eau de Toilette"                # includes the misspelling 'toillette'
    if "eau de parfum" in s or "eau de perfume" in s:
        return "Eau de Parfum"
    if "parfum" in s or "extrait" in s or "elixir" in s:
        return "Parfum / Extrait"
    if "cologne" in s or s == "edc":
        return "Cologne"
    if "perfume" in s or "fragrance" in s:
        return "Generic fragrance"
    return "Other"                              # body sprays, oils, deodorants, gift sets


df["product_type"] = df["Catatory"].apply(group_category)
print("Raw labels:", df["Catatory"].nunique(), "-> groups:", df["product_type"].nunique())
print(df["product_type"].value_counts())
log_step("Collapse category labels into 6 product types", df["Catatory"].notna().sum(), "Case, spelling and wording variants")

# 3.6 Brand: tidy whitespace; a missing brand is treated as 'Unbranded'
df["brand"] = df["Item_Name"].fillna("Unbranded").str.strip()
log_step("Missing brand set to 'Unbranded'", df["Item_Name"].isna().sum(), "Single row")

# 3.7 Location: keep the country (last part of 'City, State, Country')
df["country"] = df["itemLocation"].str.split(",").str[-1].str.strip()
df["ships_from_us"] = (df["country"] == "United States").astype(int)

# 3.8 Last update: parse dates (two month formats) and turn them into 'days since update'
dates = pd.to_datetime(df["lastUpdated"].str.replace(r" P[DS]T", "", regex=True), format="mixed", errors="coerce")
df["days_since_update"] = (dates.max() - dates).dt.total_seconds() / 86400
print("Unparsed or missing dates:", dates.isna().sum(), "| latest update:", dates.max())

# 3.9 Bottle size in ounces, parsed from the description (used for profiling, not clustering)
size = df["details"].str.extract(r"(\d+(?:\.\d+)?)\s*(?:fl\.?\s*)?oz", flags=re.IGNORECASE)[0].astype(float)
df["size_oz"] = size.where(size.between(0.3, 8.5))   # ignore implausible parses
print("Bottle size found for", df["size_oz"].notna().sum(), "listings")
```

Code A14. Notebook code cell 14

Source: COM7022 - Assignment/output/screenshots/appendix/cell14.png

COM7022_Clustering_25199053.ipynb | **code cell 15**

```
# 3.10 Transform the skewed measures. log(1 + x) keeps zero sales valid (log of 0 is undefined).
df["log_price"] = np.log1p(df["price"])
df["log_sold"] = np.log1p(df["sold"])

skew_table = pd.DataFrame({
    "skew_before": [skew(df["price"]), skew(df["sold"])],
    "skew_after_log": [skew(df["log_price"]), skew(df["log_sold"])],
}, index=["price", "sold"]).round(2)
print(skew_table)

# Outliers: count them with the 1.5 x IQR rule on the raw scale, but keep them.
# Best-selling listings are exactly what the business wants to find, so removing them would be wrong.
q1, q3 = df["sold"].quantile([0.25, 0.75])
upper_fence = q3 + 1.5 * (q3 - q1)
print("Listings above the sold upper fence (" + str(round(upper_fence)) + "):", (df["sold"] > upper_fence).sum())
log_step("Log-transform price and sold", len(df), "Outliers kept; log scale limits their pull on distances")

cleaning_table = pd.DataFrame(cleaning_log)
save_table(cleaning_table, "table03_cleaning_log.csv")
print("Rows after cleaning:", len(df))
cleaning_table
```

Code A15. Notebook code cell 15

Source: COM7022 - Assignment/output/screenshots/appendix/cell15.png

COM7022_Clustering_25199053.ipynb | **code cell 16**

```
# Figure 3: distributions after the log transform
fig, axes = plt.subplots(1, 3, figsize=(TEXT_WIDTH, 2.3))
axes[0].hist(df["log_price"], bins=35, color=MIDNIGHT_SOFT, edgecolor="white", linewidth=0.3)
panel(axes[0], "a", "log(1 + price)")
axes[1].hist(df["log_sold"], bins=35, color=TIFFANY_DEEP, edgecolor="white", linewidth=0.3)
panel(axes[1], "b", "log(1 + units sold)")
counts = df["stock_level"].value_counts().sort_index()
axes[2].bar(counts.index, counts.values, color=[TIFFANY_DEEP if v == 11 else TEAL for v in counts.index],
            edgecolor="white", linewidth=0.3, width=0.8)
axes[2].annotate("censored ceiling\n('More than 10')", xy=(10.6, counts.loc[11] * 0.9), xytext=(6.5, counts.loc[11] * 0.78),
                 fontsize=7, color=MIDNIGHT, ha="center", va="center",
                 arrowprops=dict(arrowstyle="-|>", color=MUTE, lw=0.6, mutation_scale=7))
panel(axes[2], "c", "Stock level")
axes[2].set_xticks(range(1, 12))
for ax in axes:
    ax.set_ylabel("Listings")
save_figure(fig, "fig03_transformed_distributions.png")
plt.show()
```

Code A16. Notebook code cell 16

Source: COM7022 - Assignment/output/screenshots/appendix/cell16.png

COM7022_Clustering_25199053.ipynb | **code cell 17**

```
# Descriptive statistics after cleaning (these are the numbers the models see, before scaling)
clean_desc = df[["price", "sold", "stock_level", "log_price", "log_sold", "days_since_update", "size_oz"]].describe().T
clean_desc["skewness"] = [skew(df[c].dropna()) for c in clean_desc.index]
clean_desc = clean_desc.round(2)
save_table(clean_desc, "table04_descriptive_statistics_clean.csv")
clean_desc
```

Code A17. Notebook code cell 17

Source: COM7022 - Assignment/output/screenshots/appendix/cell17.png

A · 4. Feature selection

COM7022_Clustering_25199053.ipynb | **code cell 18**

```
candidates = ["log_price", "log_sold", "stock_level", "days_since_update", "size_oz"]
corr = df[candidates].corr(method="spearman")

# Figure 4: Spearman correlation between candidate features
nice = {"log_price": "log price", "log_sold": "log units sold", "stock_level": "stock level",
        "days_since_update": "days since update", "size_oz": "bottle size (oz)"}
fig, ax = plt.subplots(figsize=(NARROW, 3.3))
sns.heatmap(corr.rename(index=nice, columns=nice), annot=True, fmt=".2f", cmap=ARDEN_DIVERGING, vmin=-1, vmax=1,
            linewidths=1, linecolor="white", square=True, annot_kws={"fontsize": 7.5},
            cbar_kws={"label": "Spearman correlation", "shrink": 0.8}, ax=ax)
ax.grid(False)
plt.setp(ax.get_xticklabels(), rotation=30, ha="right")
save_figure(fig, "fig04_feature_correlation.png")
plt.show()
```

Code A18. Notebook code cell 18

Source: COM7022 - Assignment/output/screenshots/appendix/cell18.png

COM7022_Clustering_25199053.ipynb | **code cell 19**

```
# Compare feature sets on the same footing: K-Means silhouette for k = 2..6.
# Rows with missing values in optional features are filled with the median only for this comparison.
feature_sets = {
    "A: price + sold": ["log_price", "log_sold"],
    "B: price + sold + stock": ["log_price", "log_sold", "stock_level"],
    "C: B + days since update": ["log_price", "log_sold", "stock_level", "days_since_update"],
    "D: B + bottle size": ["log_price", "log_sold", "stock_level", "size_oz"],
    "E: B + days + size": ["log_price", "log_sold", "stock_level", "days_since_update", "size_oz"],
}

rows = []
for name, cols in feature_sets.items():
    data = df[cols].fillna(df[cols].median())
    X_try = StandardScaler().fit_transform(data)
    row = {"feature_set": name, "n_features": len(cols)}
    for k in range(2, 7):
        labels_try = KMeans(n_clusters=k, n_init=20, random_state=SEED).fit_predict(X_try)
        row["k=" + str(k)] = round(silhouette_score(X_try, labels_try), 3)
    rows.append(row)

feature_scan = pd.DataFrame(rows).set_index("feature_set")
save_table(feature_scan, "table05_feature_set_scan.csv")
feature_scan
```

Code A19. Notebook code cell 19

Source: COM7022 - Assignment/output/screenshots/appendix/cell19.png

COM7022_Clustering_25199053.ipynb | **code cell 20**

```
FEATURES = ["log_price", "log_sold", "stock_level"]
scaler = StandardScaler()
X = scaler.fit_transform(df[FEATURES])
print("Feature matrix:", X.shape)
print("Means after scaling:", X.mean(axis=0).round(3))
print("Std after scaling:", X.std(axis=0).round(3))
```

Code A20. Notebook code cell 20

Source: COM7022 - Assignment/output/screenshots/appendix/cell20.png

A · 5. Principal Component Analysis

COM7022_Clustering_25199053.ipynb | **code cell 21**

```
pca = PCA(random_state=SEED)
X_pca_full = pca.fit_transform(X)
explained = pca.explained_variance_ratio_
loadings = pd.DataFrame(pca.components_.T, index=FEATURES,
                        columns=["PC" + str(i + 1) for i in range(len(FEATURES))]).round(3)
print("Explained variance ratio:", explained.round(3))
print("Cumulative:", explained.cumsum().round(3))
save_table(loadings, "table06_pca_loadings.csv")
loadings
```

Code A21. Notebook code cell 21

Source: COM7022 - Assignment/output/screenshots/appendix/cell21.png

COM7022_Clustering_25199053.ipynb | **code cell 22**

```
# Figure 5: explained variance (left) and feature loadings on PC1/PC2 (right)
X_2d = X_pca_full[:, :2]
fig, axes = plt.subplots(1, 2, figsize=(TEXT_WIDTH, 2.8), gridspec_kw={"width_ratios": [1, 1.35]})
pcs = ["PC1", "PC2", "PC3"]
axes[0].bar(pcs, explained, color=[MIDNIGHT, MIDNIGHT_SOFT, GRID], width=0.6)
axes[0].plot(pcs, explained.cumsum(), color=TIFFANY_DEEP, marker="o", label="Cumulative")
for i, v in enumerate(explained.cumsum()):
    axes[0].text(i, v + 0.04, str(round(v * 100, 1)) + "%", ha="center", color=MIDNIGHT, fontsize=7.5)
axes[0].set_ylim(0, 1.15)
axes[0].set_ylabel("Share of variance")
panel(axes[0], "a", "Explained variance")
axes[0].legend(loc="center right")

axes[1].scatter(X_2d[:, 0], X_2d[:, 1], s=4, color=NOISE_GREY, alpha=0.45, linewidths=0)
label_offset = {"log_price": (0, 0.25, "center", "bottom"), "log_sold": (0.12, 0.12, "left", "bottom"),
                "stock_level": (0.12, -0.12, "left", "top")}
for i, feature in enumerate(FEATURES):
    x, y = loadings.iloc[i, 0] * 2.5, loadings.iloc[i, 1] * 2.5
    axes[1].annotate("", xy=(x, y), xytext=(0, 0),
                     arrowprops=dict(arrowstyle="-|>", color=TIFFANY_DEEP, lw=1.3, mutation_scale=9))
    dx, dy, ha, va = label_offset[feature]
    axes[1].text(x + dx, y + dy, feature, color=MIDNIGHT, fontsize=7.5, fontweight="bold", ha=ha, va=va)
axes[1].axhline(0, color=GRID, linewidth=0.6)
axes[1].axvline(0, color=GRID, linewidth=0.6)
axes[1].set_xlabel("PC1, commercial traction (" + str(round(explained[0] * 100, 1)) + "%)")
axes[1].set_ylabel("PC2, price tier (" + str(round(explained[1] * 100, 1)) + "%)")
axes[1].grid(False)
panel(axes[1], "b", "Biplot: listings and feature directions")
save_figure(fig, "fig05_pca_variance_biplot.png")
plt.show()
```

Code A22. Notebook code cell 22

Source: COM7022 - Assignment/output/screenshots/appendix/cell22.png

A · 6. K-Means

COM7022_Clustering_25199053.ipynb | **code cell 23**

```
k_values = range(2, 11)
kmeans_scan = []
for k in k_values:
    model = KMeans(n_clusters=k, n_init=20, random_state=SEED)
    labels_k = model.fit_predict(X)
    kmeans_scan.append({
        "k": k,
        "inertia": round(model.inertia_, 1),
        "silhouette": round(silhouette_score(X, labels_k), 3),
        "davies_bouldin": round(davies_bouldin_score(X, labels_k), 3),
        "calinski_harabasz": round(calinski_harabasz_score(X, labels_k), 1),
        "smallest_cluster": int(np.bincount(labels_k).min()),
    })
kmeans_scan = pd.DataFrame(kmeans_scan).set_index("k")
save_table(kmeans_scan, "table07_kmeans_k_scan.csv")
kmeans_scan
```

Code A23. Notebook code cell 23

Source: COM7022 - Assignment/output/screenshots/appendix/cell23.png

COM7022_Clustering_25199053.ipynb | **code cell 24**

```
K_FINAL = 4
```

Code A24. Notebook code cell 24

Source: COM7022 - Assignment/output/screenshots/appendix/cell24.png

COM7022_Clustering_25199053.ipynb | **code cell 25**

```
# Figure 6: elbow (inertia) and silhouette against k
fig, ax1 = plt.subplots(figsize=(TEXT_WIDTH, 2.5))
ax1.axvspan(K_FINAL - 0.3, K_FINAL + 0.3, color=MIST, zorder=0)
l1 = ax1.plot(kmeans_scan.index, kmeans_scan["inertia"], color=MIDNIGHT, marker="o", label="Inertia (left axis)")
ax1.set_xlabel("Number of clusters, k")
ax1.set_ylabel("Inertia (within-cluster SS)")
ax1.set_xticks(list(kmeans_scan.index))
ax2 = ax1.twinx()
l2 = ax2.plot(kmeans_scan.index, kmeans_scan["silhouette"], color=TIFFANY_DEEP, marker="s",
              label="Silhouette (right axis)")
ax2.set_ylabel("Silhouette score")
ax2.grid(False)
ax2.spines["right"].set_visible(True)
ax1.text(K_FINAL, kmeans_scan["inertia"].max() * 0.98, "chosen k = " + str(K_FINAL), ha="center", va="top",
         fontsize=7.5, color=TIFFANY_DEEP, fontweight="bold")
ax1.legend(l1 + l2, [l.get_label() for l in l1 + l2], loc="upper right")
save_figure(fig, "fig06_kmeans_elbow_silhouette.png")
plt.show()
```

Code A25. Notebook code cell 25

Source: COM7022 - Assignment/output/screenshots/appendix/cell25.png

COM7022_Clustering_25199053.ipynb | **code cell 26**

```
kmeans = KMeans(n_clusters=K_FINAL, n_init=20, random_state=SEED)
df["kmeans"] = kmeans.fit_predict(X)

# Centroids back in original units, which is how a business reader thinks about them
centroids_scaled = pd.DataFrame(kmeans.cluster_centers_, columns=FEATURES)
centroids = pd.DataFrame(scaler.inverse_transform(kmeans.cluster_centers_), columns=FEATURES)
centroids["price_usd"] = np.expm1(centroids["log_price"]).round(2)
centroids["units_sold"] = np.expm1(centroids["log_sold"]).round(0)
centroids["stock_level"] = centroids["stock_level"].round(1)
centroids["listings"] = df["kmeans"].value_counts().sort_index().values
print(centroids[["price_usd", "units_sold", "stock_level", "listings"]])
```

Code A26. Notebook code cell 26

Source: COM7022 - Assignment/output/screenshots/appendix/cell26.png

COM7022_Clustering_25199053.ipynb | **code cell 27**

```
# Figure 7: K-Means clusters on the PCA plane, with a silhouette plot per cluster


def plot_clusters_pca(ax, labels, title, noise_label=None, colours=None):
    colours = colours or CLUSTER_COLOURS
    for lab in sorted(set(labels)):
        mask = labels == lab
        if noise_label is not None and lab == noise_label:
            ax.scatter(X_2d[mask, 0], X_2d[mask, 1], s=7, color=NOISE_GREY, marker="x", linewidths=0.6,
                       label="Noise")
        else:
            ax.scatter(X_2d[mask, 0], X_2d[mask, 1], s=5, alpha=0.85, linewidths=0,
                       color=colours[lab % len(colours)], label="Cluster " + str(lab))
    ax.set_xlabel("PC1, commercial traction")
    ax.set_ylabel("PC2, price tier")
    ax.grid(False)
    ax.set_title(title)
    ax.legend(loc="lower left", markerscale=2.2, handletextpad=0.3, borderaxespad=0.2)


fig, axes = plt.subplots(1, 2, figsize=(TEXT_WIDTH, 2.9))
plot_clusters_pca(axes[0], df["kmeans"].values, "(a)  K-Means (k = " + str(K_FINAL) + ") on the PCA plane")

sil_values = silhouette_samples(X, df["kmeans"])
y_lower = 10
for lab in range(K_FINAL):
    values = np.sort(sil_values[df["kmeans"].values == lab])
    y_upper = y_lower + len(values)
    axes[1].fill_betweenx(np.arange(y_lower, y_upper), 0, values, color=CLUSTER_COLOURS[lab], linewidth=0)
    axes[1].text(-0.06, y_lower + len(values) / 2, str(lab), color=MIDNIGHT, fontweight="bold", fontsize=7.5,
                 ha="right", va="center")
    y_lower = y_upper + 10
axes[1].axvline(sil_values.mean(), color=MIDNIGHT, linestyle="--", linewidth=0.9,
                label="Mean = " + str(round(sil_values.mean(), 3)))
axes[1].set_xlabel("Silhouette value")
axes[1].set_ylabel("Listings by cluster")
axes[1].set_yticks([])
axes[1].grid(axis="y", visible=False)
axes[1].grid(axis="x", visible=True)
panel(axes[1], "b", "Silhouette per listing")
axes[1].legend(loc="lower right")
save_figure(fig, "fig07_kmeans_clusters_silhouette.png")
plt.show()
```

Code A27. Notebook code cell 27

Source: COM7022 - Assignment/output/screenshots/appendix/cell27.png

A · 7. Hierarchical clustering

COM7022_Clustering_25199053.ipynb | **code cell 28**

```
linkage_methods = ["single", "complete", "average", "ward"]
distances = pdist(X)
linkage_results = {}
linkage_rows = []
for method in linkage_methods:
    Z = linkage(X, method=method)
    linkage_results[method] = Z
    coph_corr, _ = cophenet(Z, distances)
    labels_h = fcluster(Z, t=K_FINAL, criterion="maxclust")
    sizes = np.bincount(labels_h)[1:]
    linkage_rows.append({
        "linkage": method,
        "cophenetic_corr": round(coph_corr, 3),
        "silhouette_at_k": round(silhouette_score(X, labels_h), 3) if len(set(labels_h)) > 1 else np.nan,
        "cluster_sizes": "/".join(str(s) for s in sorted(sizes, reverse=True)),
    })
linkage_table = pd.DataFrame(linkage_rows).set_index("linkage")
save_table(linkage_table, "table08_linkage_comparison.csv")
linkage_table
```

Code A28. Notebook code cell 28

Source: COM7022 - Assignment/output/screenshots/appendix/cell28.png

COM7022_Clustering_25199053.ipynb | **code cell 29**

```
# Figure 8: dendrograms for the four linkages (truncated to the last 30 merges for readability)
with plt.rc_context({"lines.linewidth": 0.8}):
    fig, axes = plt.subplots(2, 2, figsize=(TEXT_WIDTH, 4.2))
    for ax, letter, method in zip(axes.ravel(), "abcd", linkage_methods):
        dendrogram(linkage_results[method], truncate_mode="lastp", p=30, ax=ax,
                   color_threshold=0, above_threshold_color=MIDNIGHT_SOFT, no_labels=True)
        panel(ax, letter, method.capitalize() + " linkage (cophenetic r = "
              + str(linkage_table.loc[method, "cophenetic_corr"]) + ")")
        ax.set_ylabel("Merge distance")
        ax.grid(axis="x", visible=False)
    save_figure(fig, "fig08_dendrograms_four_linkages.png")
    plt.show()
```

Code A29. Notebook code cell 29

Source: COM7022 - Assignment/output/screenshots/appendix/cell29.png

COM7022_Clustering_25199053.ipynb | **code cell 30**

```
# Ward: metrics across k, then the final cut at the same k as K-Means for a fair comparison
Z_ward = linkage_results["ward"]
ward_scan = []
for k in range(2, 11):
    labels_w = fcluster(Z_ward, t=k, criterion="maxclust")
    ward_scan.append({"k": k,
                      "silhouette": round(silhouette_score(X, labels_w), 3),
                      "davies_bouldin": round(davies_bouldin_score(X, labels_w), 3),
                      "calinski_harabasz": round(calinski_harabasz_score(X, labels_w), 1)})
ward_scan = pd.DataFrame(ward_scan).set_index("k")
save_table(ward_scan, "table09_ward_k_scan.csv")

# scikit-learn's AgglomerativeClustering gives the same Ward tree with 0-based labels
ward_model = AgglomerativeClustering(n_clusters=K_FINAL, linkage="ward")
df["ward"] = ward_model.fit_predict(X)
print("Ward cluster sizes:", np.bincount(df["ward"]))
print("Agreement with K-Means (Adjusted Rand Index):", round(adjusted_rand_score(df["kmeans"], df["ward"]), 3))
ward_scan
```

Code A30. Notebook code cell 30

Source: COM7022 - Assignment/output/screenshots/appendix/cell30.png

COM7022_Clustering_25199053.ipynb | **code cell 31**

```
# Figure 9: Ward dendrogram with the cut that gives K_FINAL clusters
cut_height = (Z_ward[-K_FINAL, 2] + Z_ward[-K_FINAL + 1, 2]) / 2
set_link_color_palette([CLUSTER_COLOURS[0], CLUSTER_COLOURS[1], CLUSTER_COLOURS[2], CLUSTER_COLOURS[3]])
with plt.rc_context({"lines.linewidth": 0.9}):
    fig, ax = plt.subplots(figsize=(TEXT_WIDTH, 2.6))
    dendrogram(Z_ward, truncate_mode="lastp", p=40, ax=ax, color_threshold=cut_height,
               above_threshold_color=MIDNIGHT, no_labels=True)
    ax.axhline(cut_height, color=MIDNIGHT, linestyle="--", linewidth=0.9)
    ax.text(ax.get_xlim()[1] * 0.995, cut_height + 0.8, "cut at " + str(round(cut_height, 1)) + " -> "
            + str(K_FINAL) + " clusters", color=MIDNIGHT, ha="right", va="bottom", fontsize=7.5, fontweight="bold")
    ax.set_ylabel("Merge distance (Ward)")
    ax.grid(axis="x", visible=False)
    save_figure(fig, "fig09_ward_dendrogram_cut.png")
    plt.show()
set_link_color_palette(None)
```

Code A31. Notebook code cell 31

Source: COM7022 - Assignment/output/screenshots/appendix/cell31.png

A · 8. DBSCAN

COM7022_Clustering_25199053.ipynb | **code cell 32**

```
MIN_SAMPLES = 2 * len(FEATURES)
neighbours = NearestNeighbors(n_neighbors=MIN_SAMPLES).fit(X)
dist_k, _ = neighbours.kneighbors(X)
k_distance = np.sort(dist_k[:, -1])

# Figure 10: k-distance plot used to pick eps
fig, ax = plt.subplots(figsize=(TEXT_WIDTH, 2.3))
ax.plot(k_distance, color=MIDNIGHT)
for q, colour in [(0.90, TIFFANY_DEEP), (0.95, "#C99A2E")]:
    ax.axhline(np.quantile(k_distance, q), color=colour, linestyle="--", linewidth=0.9,
               label=str(int(q * 100)) + "th percentile = " + str(round(np.quantile(k_distance, q), 2)))
ax.set_xlabel("Listings sorted by distance")
ax.set_ylabel("Distance to " + str(MIN_SAMPLES) + "th neighbour")
ax.legend(loc="upper left")
save_figure(fig, "fig10_dbscan_k_distance.png")
plt.show()
```

Code A32. Notebook code cell 32

Source: COM7022 - Assignment/output/screenshots/appendix/cell32.png

COM7022_Clustering_25199053.ipynb | **code cell 33**

```
# Grid search over eps and min_samples. Silhouette is computed on clustered points only,
# so it must be read together with the noise share and the number of clusters.
dbscan_rows = []
for eps in [0.25, 0.30, 0.35, 0.40, 0.45, 0.50, 0.60, 0.70]:
    for min_s in [4, 6, 10, 15]:
        labels_d = DBSCAN(eps=eps, min_samples=min_s).fit_predict(X)
        n_clusters = len(set(labels_d)) - (1 if -1 in labels_d else 0)
        noise_share = (labels_d == -1).mean()
        mask = labels_d != -1
        sil = np.nan
        if n_clusters >= 2:
            sil = silhouette_score(X[mask], labels_d[mask])
        dbscan_rows.append({"eps": eps, "min_samples": min_s, "clusters": n_clusters,
                            "noise_pct": round(noise_share * 100, 1),
                            "silhouette_clustered": round(sil, 3) if not np.isnan(sil) else np.nan})
dbscan_grid = pd.DataFrame(dbscan_rows)
save_table(dbscan_grid.set_index(["eps", "min_samples"]), "table10_dbscan_grid.csv")
dbscan_grid.pivot(index="eps", columns="min_samples", values="clusters")
```

Code A33. Notebook code cell 33

Source: COM7022 - Assignment/output/screenshots/appendix/cell33.png

COM7022_Clustering_25199053.ipynb | **code cell 34**

```
dbscan_grid.pivot(index="eps", columns="min_samples", values="noise_pct")
```

Code A34. Notebook code cell 34

Source: COM7022 - Assignment/output/screenshots/appendix/cell34.png

COM7022_Clustering_25199053.ipynb | **code cell 35**

```
EPS_FINAL = 0.45
dbscan = DBSCAN(eps=EPS_FINAL, min_samples=MIN_SAMPLES)
df["dbscan"] = dbscan.fit_predict(X)
print("DBSCAN eps =", EPS_FINAL, "min_samples =", MIN_SAMPLES)
print(df["dbscan"].value_counts().sort_index())

# What do the noise points look like?  (They are the listings DBSCAN considers unusual.)
noise_profile = df.groupby(df["dbscan"] == -1)[["price", "sold", "stock_level"]].median()
noise_profile.index = ["clustered", "noise"]
noise_profile
```

Code A35. Notebook code cell 35

Source: COM7022 - Assignment/output/screenshots/appendix/cell35.png

COM7022_Clustering_25199053.ipynb | **code cell 36**

```
# Figure 11: DBSCAN result on the PCA plane
fig, ax = plt.subplots(figsize=(NARROW, 3.0))
plot_clusters_pca(ax, df["dbscan"].values, "", noise_label=-1, colours=[MIDNIGHT_SOFT, "#C8553D"])
save_figure(fig, "fig11_dbscan_clusters.png")
plt.show()
```

Code A36. Notebook code cell 36

Source: COM7022 - Assignment/output/screenshots/appendix/cell36.png

A · 9. HDBSCAN

COM7022_Clustering_25199053.ipynb | **code cell 37**

```
# Grid over both settings. min_samples = 6 matches DBSCAN, so the two density methods are comparable.
hdbscan_rows = []
for mcs in [10, 20, 30, 50, 100]:
    for min_s in [3, 6, 10]:
        labels_h = HDBSCAN(min_cluster_size=mcs, min_samples=min_s).fit_predict(X)
        n_clusters = len(set(labels_h)) - (1 if -1 in labels_h else 0)
        mask = labels_h != -1
        sil = silhouette_score(X[mask], labels_h[mask]) if n_clusters >= 2 else np.nan
        hdbscan_rows.append({"min_cluster_size": mcs, "min_samples": min_s, "clusters": n_clusters,
                             "noise_pct": round((~mask).mean() * 100, 1),
                             "silhouette_clustered": round(sil, 3) if not np.isnan(sil) else np.nan})
hdbscan_grid = pd.DataFrame(hdbscan_rows)
save_table(hdbscan_grid.set_index(["min_cluster_size", "min_samples"]), "table15_hdbscan_grid.csv")
hdbscan_grid.pivot(index="min_cluster_size", columns="min_samples", values="noise_pct")
```

Code A37. Notebook code cell 37

Source: COM7022 - Assignment/output/screenshots/appendix/cell37.png

COM7022_Clustering_25199053.ipynb | **code cell 38**

```
MIN_CLUSTER_SIZE = 30   # about 3% of listings: the smallest group worth its own business action
hdbscan = HDBSCAN(min_cluster_size=MIN_CLUSTER_SIZE, min_samples=MIN_SAMPLES)
df["hdbscan"] = hdbscan.fit_predict(X)
df["hdbscan_strength"] = hdbscan.probabilities_   # membership strength (0 = noise, 1 = cluster core)
print("HDBSCAN min_cluster_size =", MIN_CLUSTER_SIZE, "min_samples =", MIN_SAMPLES)
print(df["hdbscan"].value_counts().sort_index())

# Profile of each HDBSCAN group. The key column is the share of listings sitting on the stock ceiling
# (11 = "More than 10 available"), which is where the marketplace censors stock.
hdbscan_profile = df.groupby("hdbscan").agg(
    listings=("price", "size"),
    median_price=("price", "median"),
    median_units_sold=("sold", "median"),
    median_stock_level=("stock_level", "median"),
    pct_at_stock_ceiling=("stock_level", lambda s: round((s == 11).mean() * 100, 1)),
)
hdbscan_profile.index = ["Noise" if i == -1 else "Cluster " + str(i) for i in hdbscan_profile.index]
save_table(hdbscan_profile, "table16_hdbscan_profile.csv")
print("Share of all listings at the stock ceiling:", round((df["stock_level"] == 11).mean() * 100, 1), "%")
hdbscan_profile
```

Code A38. Notebook code cell 38

Source: COM7022 - Assignment/output/screenshots/appendix/cell38.png

COM7022_Clustering_25199053.ipynb | **code cell 39**

```
# Robustness: is the result produced by ties on the integer stock scale, or by the stock feature at all?
rng = np.random.default_rng(SEED)
jitter_runs = []
for run in range(10):
    jittered = df[FEATURES].copy()
    jittered["stock_level"] = jittered["stock_level"] + rng.uniform(-0.5, 0.5, len(df))   # break the ties
    lab = HDBSCAN(min_cluster_size=MIN_CLUSTER_SIZE, min_samples=MIN_SAMPLES).fit_predict(
        StandardScaler().fit_transform(jittered))
    jitter_runs.append({"clusters": len(set(lab)) - (1 if -1 in lab else 0), "noise_pct": (lab == -1).mean() * 100})
jitter_runs = pd.DataFrame(jitter_runs)
lab_no_stock = HDBSCAN(min_cluster_size=MIN_CLUSTER_SIZE, min_samples=MIN_SAMPLES).fit_predict(
    StandardScaler().fit_transform(df[["log_price", "log_sold"]]))
print("Stock jittered by +/-0.5 (10 runs): clusters", sorted(jitter_runs["clusters"].unique()),
      "| mean noise", round(jitter_runs["noise_pct"].mean(), 1), "%")
print("Price and sales only: clusters", len(set(lab_no_stock)) - (1 if -1 in lab_no_stock else 0),
      "| noise", round((lab_no_stock == -1).mean() * 100, 1), "%")
```

Code A39. Notebook code cell 39

Source: COM7022 - Assignment/output/screenshots/appendix/cell39.png

COM7022_Clustering_25199053.ipynb | **code cell 40**

```
# Figure 18: HDBSCAN on the PCA plane, and how its clusters line up with the stock ceiling
fig, axes = plt.subplots(1, 2, figsize=(TEXT_WIDTH, 2.9), gridspec_kw={"width_ratios": [1.25, 1]})
plot_clusters_pca(axes[0], df["hdbscan"].values, "(a)  HDBSCAN on the PCA plane", noise_label=-1,
                  colours=[MIDNIGHT_SOFT, "#C8553D"])
ceiling = hdbscan_profile["pct_at_stock_ceiling"]
bar_colours = [NOISE_GREY if name == "Noise" else [MIDNIGHT_SOFT, "#C8553D"][int(name.split()[-1])]
               for name in ceiling.index]
bars = axes[1].bar(ceiling.index, ceiling.values, color=bar_colours, width=0.6)
overall = (df["stock_level"] == 11).mean() * 100
axes[1].axhline(overall, color=TIFFANY_DEEP, linestyle="--", linewidth=0.9,
                label="All listings: " + str(round(overall, 1)) + "%")
for bar, v, n in zip(bars, ceiling.values, hdbscan_profile["listings"].values):
    axes[1].text(bar.get_x() + bar.get_width() / 2, v + 3, str(v) + "%\nn = " + str(n), ha="center", va="bottom",
                 fontsize=7, color=MIDNIGHT)
axes[1].set_ylim(0, 118)
axes[1].set_ylabel("Listings at stock ceiling (%)")
panel(axes[1], "b", "Groups vs the censored stock ceiling")
axes[1].legend(loc="upper right")
save_figure(fig, "fig18_hdbscan.png")
plt.show()
```

Code A40. Notebook code cell 40

Source: COM7022 - Assignment/output/screenshots/appendix/cell40.png

A · 10. TabClustPFN (tabular foundation model)

COM7022_Clustering_25199053.ipynb | **code cell 41**

```
import sys, subprocess, importlib, importlib.util, tempfile, time

TCP_DIR = Path(os.environ.get("TABCLUSTPFN_DIR", "TabClustPFN"))
TCP_CKPT = Path(os.environ.get("TABCLUSTPFN_CKPT", str(TCP_DIR / "tabclustpfn.ckpt")))
TCP_CKPT_ID = "1GzZBgVXw0B8y9JM89xa7ln-la6v2sUO3"          # official checkpoint (Google Drive, from the README)
TCP_SAVED = OUTPUT_DIR / "tabclustpfn_labels.csv"          # labels from the assessed run (fallback)


def ensure_rpy2_placeholder():
    # rpy2 is only used to synthesise pre-training data; inference does not call it
    try:
        importlib.import_module("rpy2.robjects")
        return
    except Exception:
        pass
    stub = Path(tempfile.gettempdir()) / "tabclustpfn_stubs" / "rpy2" / "robjects"
    stub.mkdir(parents=True, exist_ok=True)
    (stub.parent / "__init__.py").write_text("")
    (stub / "__init__.py").write_text(
        "class _Unavailable:\n"
        "    def __getattr__(self, name):\n"
        "        raise RuntimeError('rpy2/R is only needed to generate TabClustPFN pre-training data')\n"
        "    def __call__(self, *a, **k):\n"
        "        raise RuntimeError('rpy2/R is only needed to generate TabClustPFN pre-training data')\n"
        "r = _Unavailable()\nNULL = None\ndefault_converter = None\n"
        "FloatVector = IntVector = StrVector = _Unavailable()\n")
    (stub / "packages.py").write_text("from . import _Unavailable as importr\n")
    (stub / "conversion.py").write_text("from . import _Unavailable as localconverter\n")
    sys.path.insert(0, str(stub.parent.parent))


tcp_model = None
try:
    if not TCP_DIR.exists():
        subprocess.run(["git", "clone", "--depth", "1", "https://github.com/Tianqi-Zhao/TabClustPFN.git", str(TCP_DIR)],
                       check=True)
    for package in ["torch", "einops", "transformers", "tensorboard", "psutil", "tqdm"]:
        if importlib.util.find_spec(package) is None:
            subprocess.run([sys.executable, "-m", "pip", "install", "-q", package], check=True)
    if not TCP_CKPT.exists():
        if importlib.util.find_spec("gdown") is None:
            subprocess.run([sys.executable, "-m", "pip", "install", "-q", "gdown"], check=True)
        subprocess.run([sys.executable, "-m", "gdown", TCP_CKPT_ID, "-O", str(TCP_CKPT)], check=True)
    ensure_rpy2_placeholder()
    sys.path.insert(0, str(TCP_DIR))
    import torch
    from cluster import load_model as tcp_load_model, cluster as tcp_cluster
    torch.manual_seed(SEED)
    TCP_DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
    tcp_model = tcp_load_model(str(TCP_CKPT), TCP_DEVICE)
    print("TabClustPFN loaded on", TCP_DEVICE)
except Exception as error:
    print("TabClustPFN could not be loaded here (" + type(error).__name__ + ": " + str(error)[:120] + ").")
    print("Falling back to the labels saved from the assessed run:", TCP_SAVED)


def fit_tabclustpfn(data, k=None):
    labels, k_used = tcp_cluster(tcp_model, np.asarray(data), num_clusters=k, device=TCP_DEVICE)
    return np.asarray(labels)
```

Code A41. Notebook code cell 41

Source: COM7022 - Assignment/output/screenshots/appendix/cell41.png

COM7022_Clustering_25199053.ipynb | **code cell 42**

```
if tcp_model is not None:
    start = time.time()
    labels_auto, k_auto = tcp_cluster(tcp_model, X, num_clusters=None, device=TCP_DEVICE)
    seconds_auto = time.time() - start
    labels_k4 = fit_tabclustpfn(X, K_FINAL)
    df["tabclustpfn"] = np.asarray(labels_auto)
    df["tabclustpfn_k4"] = labels_k4
    df[["tabclustpfn", "tabclustpfn_k4"]].to_csv(TCP_SAVED, index=False)
    print("Automatic k predicted by the model:", k_auto, "| inference time:", round(seconds_auto, 1), "s")
else:
    saved = pd.read_csv(TCP_SAVED)
    df["tabclustpfn"], df["tabclustpfn_k4"] = saved["tabclustpfn"].values, saved["tabclustpfn_k4"].values
    k_auto = df["tabclustpfn"].nunique()

# Two robustness checks specific to an in-context model: does k stay the same on subsamples,
# and does the order of the rows matter?
tcp_checks = {}
if tcp_model is not None:
    rng = np.random.default_rng(SEED)
    sub_k = []
    for run in range(30):
        idx = rng.choice(len(X), size=int(0.8 * len(X)), replace=False)
        sub_k.append(tcp_cluster(tcp_model, X[idx], num_clusters=None, device=TCP_DEVICE)[1])
    perm = rng.permutation(len(X))
    shuffled = fit_tabclustpfn(X[perm])
    restored = np.empty_like(shuffled)
    restored[perm] = shuffled
    tcp_checks = {"predicted_k": int(k_auto),
                  "subsamples_with_same_k": int(sum(k == k_auto for k in sub_k)),
                  "ari_row_shuffle": round(adjusted_rand_score(df["tabclustpfn"], restored), 3)}
    print("Same k on", tcp_checks["subsamples_with_same_k"], "of 30 subsamples | ARI after shuffling rows:",
          tcp_checks["ari_row_shuffle"])

tcp_profile_rows = []
for column, variant in [("tabclustpfn", "auto k = " + str(k_auto)), ("tabclustpfn_k4", "k = " + str(K_FINAL))]:
    for lab, group in df.groupby(column):
        tcp_profile_rows.append({
            "variant": variant, "cluster": lab, "listings": len(group),
            "median_price": round(group["price"].median(), 2),
            "median_units_sold": group["sold"].median(),
            "share_of_units_sold_pct": round(group["sold"].sum() / df["sold"].sum() * 100, 1),
            "median_stock_level": group["stock_level"].median(),
            "pct_at_stock_ceiling": round((group["stock_level"] == 11).mean() * 100, 1)})
tcp_profile = pd.DataFrame(tcp_profile_rows).set_index(["variant", "cluster"])
save_table(tcp_profile, "table17_tabclustpfn.csv")
print("ARI with K-Means: auto", round(adjusted_rand_score(df["kmeans"], df["tabclustpfn"]), 3),
      "| k = 4", round(adjusted_rand_score(df["kmeans"], df["tabclustpfn_k4"]), 3))
tcp_profile
```

Code A42. Notebook code cell 42

Source: COM7022 - Assignment/output/screenshots/appendix/cell42.png

COM7022_Clustering_25199053.ipynb | **code cell 43**

```
# Figure 19: TabClustPFN partitions on the PCA plane, and how the k = 4 version maps onto K-Means
fig, axes = plt.subplots(1, 3, figsize=(TEXT_WIDTH, 2.6), gridspec_kw={"width_ratios": [1, 1, 0.9]})
tcp_colours = [MIDNIGHT_SOFT, "#8FA3B1", TIFFANY, "#7D6B9E"]   # neutral: segment colours stay reserved for K-Means
plot_clusters_pca(axes[0], df["tabclustpfn"].values, "(a)  Automatic k = " + str(k_auto), colours=tcp_colours)
plot_clusters_pca(axes[1], df["tabclustpfn_k4"].values, "(b)  Fixed k = " + str(K_FINAL), colours=tcp_colours)
cross = pd.crosstab(df["tabclustpfn_k4"], df["kmeans"])
sns.heatmap(cross, annot=True, fmt="d", cmap=ARDEN_CMAP, cbar=False, linewidths=1, linecolor="white",
            annot_kws={"fontsize": 7}, ax=axes[2])
axes[2].set_xlabel("K-Means cluster")
axes[2].set_ylabel("TabClustPFN (k = 4)")
axes[2].grid(False)
panel(axes[2], "c", "Overlap with K-Means")
for ax in axes[:2]:
    ax.legend(loc="lower left", markerscale=2, fontsize=6.5, handletextpad=0.2, borderaxespad=0.1)
save_figure(fig, "fig19_tabclustpfn.png")
plt.show()
```

Code A43. Notebook code cell 43

Source: COM7022 - Assignment/output/screenshots/appendix/cell43.png

A · 11. Model comparison and validation

COM7022_Clustering_25199053.ipynb | **code cell 44**

```python
def internal_scores(labels, ignore_noise=False):
    labels = np.asarray(labels)
    mask = np.ones(len(labels), dtype=bool)
    if ignore_noise:
        mask = labels != -1
    return {
        "silhouette": round(silhouette_score(X[mask], labels[mask]), 3),
        "davies_bouldin": round(davies_bouldin_score(X[mask], labels[mask]), 3),
        "calinski_harabasz": round(calinski_harabasz_score(X[mask], labels[mask]), 1),
    }


def bootstrap_stability(fit_function, n_runs=30, frac=0.8):
    # Fit on the full data once, then on random subsamples, and compare labels on the shared rows
    full_labels = fit_function(X)
    rng = np.random.default_rng(SEED)
    scores = []
    for run in range(n_runs):
        idx = rng.choice(len(X), size=int(frac * len(X)), replace=False)
        sub_labels = fit_function(X[idx])
        scores.append(adjusted_rand_score(full_labels[idx], sub_labels))
    return np.mean(scores), np.std(scores)


def fit_kmeans(data):
    return KMeans(n_clusters=K_FINAL, n_init=20, random_state=SEED).fit_predict(data)


def fit_ward(data):
    return AgglomerativeClustering(n_clusters=K_FINAL, linkage="ward").fit_predict(data)


def fit_dbscan(data):
    return DBSCAN(eps=EPS_FINAL, min_samples=MIN_SAMPLES).fit_predict(data)


def fit_hdbscan(data):
    return HDBSCAN(min_cluster_size=MIN_CLUSTER_SIZE, min_samples=MIN_SAMPLES).fit_predict(data)


comparison_rows = []
for name, column, fit_function, ignore_noise in [
        ("K-Means", "kmeans", fit_kmeans, False),
        ("Hierarchical (Ward)", "ward", fit_ward, False),
        ("DBSCAN", "dbscan", fit_dbscan, True),
        ("HDBSCAN", "hdbscan", fit_hdbscan, True),
        ("TabClustPFN (auto k)", "tabclustpfn", fit_tabclustpfn if tcp_model is not None else None, False),
        ("TabClustPFN (k = 4)", "tabclustpfn_k4",
         (lambda data: fit_tabclustpfn(data, K_FINAL)) if tcp_model is not None else None, False)]:
    labels = df[column].values
    scores = internal_scores(labels, ignore_noise=ignore_noise)
    if fit_function is not None:
        stab_mean, stab_std = bootstrap_stability(fit_function)
    else:      # TabClustPFN unavailable in this environment: reuse the stability measured in the assessed run
        previous = pd.read_csv(TABLE_DIR / "table11_model_comparison.csv", index_col=0)
        stab_mean, stab_std = previous.loc[name, "stability_ARI_mean"], previous.loc[name, "stability_ARI_sd"]
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    sizes = pd.Series(labels[labels != -1]).value_counts()
    row = {"model": name, "clusters": n_clusters, "noise_pct": round((labels == -1).mean() * 100, 1)}
    row.update(scores)
    row["stability_ARI_mean"] = round(stab_mean, 3)
    row["stability_ARI_sd"] = round(stab_std, 3)
    row["smallest_cluster_pct"] = round(sizes.min() / len(labels) * 100, 1)
    row["largest_cluster_pct"] = round(sizes.max() / len(labels) * 100, 1)
    comparison_rows.append(row)

comparison = pd.DataFrame(comparison_rows).set_index("model")
save_table(comparison, "table11_model_comparison.csv")
comparison
```

Code A44. Notebook code cell 44

Source: COM7022 - Assignment/output/screenshots/appendix/cell44.png

COM7022_Clustering_25199053.ipynb | **code cell 45**

```
# Seed stability for K-Means: does the answer change with the random start?
seed_labels = [KMeans(n_clusters=K_FINAL, n_init=20, random_state=s).fit_predict(X) for s in range(10)]
seed_ari = [adjusted_rand_score(seed_labels[0], lab) for lab in seed_labels[1:]]
print("K-Means ARI across 10 seeds: min", round(min(seed_ari), 3), "mean", round(np.mean(seed_ari), 3))

# Sensitivity 1: min-max scaling instead of z-scores (as used in the lessons)
X_minmax = MinMaxScaler().fit_transform(df[FEATURES])
km_minmax = KMeans(n_clusters=K_FINAL, n_init=20, random_state=SEED).fit_predict(X_minmax)
print("ARI z-score vs min-max K-Means:", round(adjusted_rand_score(df["kmeans"], km_minmax), 3))

# Sensitivity 2: drop the top 1% sellers and re-fit; compare labels on the remaining rows
keep = df["sold"] < df["sold"].quantile(0.99)
X_trim = StandardScaler().fit_transform(df.loc[keep, FEATURES])
km_trim = KMeans(n_clusters=K_FINAL, n_init=20, random_state=SEED).fit_predict(X_trim)
print("ARI full vs without top 1% sellers:", round(adjusted_rand_score(df.loc[keep, "kmeans"], km_trim), 3))

sensitivity = pd.DataFrame({
    "check": ["K-Means across 10 seeds (mean ARI)", "z-score vs min-max scaling (ARI)", "with vs without top 1% sellers (ARI)"],
    "ARI": [round(np.mean(seed_ari), 3),
            round(adjusted_rand_score(df["kmeans"], km_minmax), 3),
            round(adjusted_rand_score(df.loc[keep, "kmeans"], km_trim), 3)],
}).set_index("check")
save_table(sensitivity, "table12_sensitivity_checks.csv")
sensitivity
```

Code A45. Notebook code cell 45

Source: COM7022 - Assignment/output/screenshots/appendix/cell45.png

COM7022_Clustering_25199053.ipynb | **code cell 46**

```
# Figure 12: side-by-side comparison of the models
metrics_to_plot = [("silhouette", "Silhouette (higher is better)"),
                   ("davies_bouldin", "Davies-Bouldin (lower is better)"),
                   ("stability_ARI_mean", "Stability ARI (higher is better)")]
short = {"Hierarchical (Ward)": "Ward", "TabClustPFN (auto k)": "TCP auto", "TabClustPFN (k = 4)": "TCP k=4"}
fig, axes = plt.subplots(1, 3, figsize=(TEXT_WIDTH, 2.9))
for ax, letter, (metric, label) in zip(axes, "abc", metrics_to_plot):
    values = comparison[metric]
    names = [short.get(m, m) for m in comparison.index]
    bars = ax.bar(names, values, color=[MODEL_COLOURS[m] for m in comparison.index], width=0.65,
                  edgecolor="white", linewidth=0.4)
    for bar, m in zip(bars, comparison.index):
        if comparison.loc[m, "noise_pct"] > 0:      # density models: scored on clustered points only
            bar.set_hatch("////")
    for bar, v in zip(bars, values):
        ax.text(bar.get_x() + bar.get_width() / 2, max(v, 0) + max(values) * 0.03, format(v, ".3f"), ha="center",
                va="bottom", color=MIDNIGHT, fontsize=7)
    panel(ax, letter, label)
    ax.set_ylim(min(0, values.min() * 1.3), max(values) * 1.22)
    ax.axhline(0, color="#9AAAB4", linewidth=0.6)
    plt.setp(ax.get_xticklabels(), rotation=40, ha="right")
fig.supxlabel("Hatched bars: DBSCAN and HDBSCAN are scored on clustered listings only ("
              + ", ".join(str(comparison.loc[m, "noise_pct"]) + "% noise for " + m
                          for m in comparison.index if comparison.loc[m, "noise_pct"] > 0) + ").",
              fontsize=7, color=MUTE)
save_figure(fig, "fig12_model_comparison.png")
plt.show()
```

Code A46. Notebook code cell 46

Source: COM7022 - Assignment/output/screenshots/appendix/cell46.png

A · 12. Final model and cluster profiles

COM7022_Clustering_25199053.ipynb | **code cell 47**

```
FINAL = "kmeans"
groups = df.groupby(FINAL)
profile = pd.DataFrame({
    "listings": groups.size(),
    "share_of_listings_pct": (groups.size() / len(df) * 100).round(1),
    "median_price_usd": groups["price"].median().round(2),
    "median_units_sold": groups["sold"].median().round(0),
    "mean_units_sold": groups["sold"].mean().round(0),
    "share_of_units_sold_pct": (groups["sold"].sum() / df["sold"].sum() * 100).round(1),
    "median_stock_level": groups["stock_level"].median(),
    "pct_last_one": groups["stock_level"].apply(lambda s: (s == 1).mean() * 100).round(1),
    "pct_ships_from_us": (groups["ships_from_us"].mean() * 100).round(1),
    "median_days_since_update": groups["days_since_update"].median().round(1),
})
top_type = groups["product_type"].agg(lambda s: s.value_counts().index[0])
top_type_share = groups["product_type"].agg(lambda s: round(s.value_counts(normalize=True).iloc[0] * 100, 1))
profile["top_product_type"] = top_type + " (" + top_type_share.astype(str) + "%)"
profile["top_brands"] = groups["brand"].agg(lambda s: ", ".join(s.value_counts().index[:3]))
save_table(profile, "table13_cluster_profiles.csv")
profile.T
```

Code A47. Notebook code cell 47

Source: COM7022 - Assignment/output/screenshots/appendix/cell47.png

COM7022_Clustering_25199053.ipynb | **code cell 48**

```
# Product-type mix per cluster (row percentages)
type_mix = pd.crosstab(df[FINAL], df["product_type"], normalize="index").mul(100).round(1)
save_table(type_mix, "table14_product_type_mix.csv")
type_mix
```

Code A48. Notebook code cell 48

Source: COM7022 - Assignment/output/screenshots/appendix/cell48.png

COM7022_Clustering_25199053.ipynb | **code cell 49**

```
# Figure 13: cluster profiles as standardised feature means (how each cluster differs from the average listing)
z_profile = pd.DataFrame(X, columns=["Price (log)", "Units sold (log)", "Stock level"])
z_profile[FINAL] = df[FINAL].values
z_means = z_profile.groupby(FINAL).mean()
z_means.index = ["Cluster " + str(i) for i in z_means.index]

fig, axes = plt.subplots(1, 2, figsize=(TEXT_WIDTH, 2.6), gridspec_kw={"width_ratios": [1.1, 1]})
sns.heatmap(z_means, annot=True, fmt=".2f", cmap=ARDEN_DIVERGING, center=0, vmin=-2, vmax=2,
            linewidths=1, linecolor="white", annot_kws={"fontsize": 7.5},
            cbar_kws={"label": "Mean z-score", "shrink": 0.85}, ax=axes[0])
axes[0].grid(False)
axes[0].set_ylabel("")
panel(axes[0], "a", "Centres relative to the average listing")

share = profile[["share_of_listings_pct", "share_of_units_sold_pct"]].copy()
x_pos = np.arange(len(share))
for i, lab in enumerate(share.index):
    axes[1].bar(i - 0.2, share.loc[lab, "share_of_listings_pct"], width=0.4, color=CLUSTER_COLOURS[lab], alpha=0.35)
    axes[1].bar(i + 0.2, share.loc[lab, "share_of_units_sold_pct"], width=0.4, color=CLUSTER_COLOURS[lab])
from matplotlib.patches import Patch
axes[1].legend(handles=[Patch(facecolor=MIDNIGHT_SOFT, alpha=0.35, label="Share of listings (pale)"),
                        Patch(facecolor=MIDNIGHT_SOFT, label="Share of units sold (solid)")], loc="upper left")
axes[1].set_xticks(x_pos)
axes[1].set_xticklabels(["Cluster " + str(i) for i in share.index])
axes[1].set_ylabel("Percent")
panel(axes[1], "b", "Listings vs units sold")
save_figure(fig, "fig13_cluster_profiles.png")
plt.show()
```

Code A49. Notebook code cell 49

Source: COM7022 - Assignment/output/screenshots/appendix/cell49.png

COM7022_Clustering_25199053.ipynb | **code cell 50**

```
# Figure 14: distributions of price, units sold and stock inside each cluster
fig, axes = plt.subplots(1, 3, figsize=(TEXT_WIDTH, 2.4))
order = sorted(df[FINAL].unique())
palette = [CLUSTER_COLOURS[i] for i in order]
box_style = dict(order=order, palette=palette, hue=FINAL, legend=False, fliersize=1.5, linewidth=0.7, width=0.6)
sns.boxplot(data=df, x=FINAL, y="price", ax=axes[0], **box_style)
panel(axes[0], "a", "Price")
axes[0].set_ylabel("Price (US$)")
sns.boxplot(data=df, x=FINAL, y="sold", ax=axes[1], **box_style)
axes[1].set_yscale("log")
panel(axes[1], "b", "Units sold (log scale)")
axes[1].set_ylabel("Units sold")
sns.boxplot(data=df, x=FINAL, y="stock_level", ax=axes[2], **box_style)
panel(axes[2], "c", "Stock level")
axes[2].set_ylabel("Stock level (11 = more than 10)")
for ax in axes:
    ax.set_xlabel("Cluster")
save_figure(fig, "fig14_cluster_distributions.png")
plt.show()
```

Code A50. Notebook code cell 50

Source: COM7022 - Assignment/output/screenshots/appendix/cell50.png

COM7022_Clustering_25199053.ipynb | **code cell 51**

```
# Save the frozen results ledger: cluster labels for every listing plus the headline numbers
df[["Item_Name", "details", "product_type", "price", "sold", "stock_level", "stock_imputed",
    "country", "kmeans", "ward", "dbscan", "hdbscan", "tabclustpfn", "tabclustpfn_k4"]].to_csv(OUTPUT_DIR / "cluster_assignments.csv", index=False)

ledger = {
    "rows_raw": int(len(raw)),
    "rows_clean": int(len(df)),
    "features": FEATURES,
    "k_final": K_FINAL,
    "dbscan_eps": EPS_FINAL,
    "dbscan_min_samples": MIN_SAMPLES,
    "hdbscan_min_cluster_size": MIN_CLUSTER_SIZE,
    "hdbscan_min_samples": MIN_SAMPLES,
    "tabclustpfn_checks": tcp_checks,
    "pca_explained_pc1_pc2": [round(float(v), 3) for v in explained[:2]],
    "comparison": comparison.reset_index().to_dict(orient="records"),
    "sensitivity": sensitivity.reset_index().to_dict(orient="records"),
    "kmeans_ward_ari": round(adjusted_rand_score(df["kmeans"], df["ward"]), 3),
    "profiles": profile.reset_index().to_dict(orient="records"),
}
with open(OUTPUT_DIR / "results_ledger.json", "w") as f:
    json.dump(ledger, f, indent=2, default=str)
print("Results ledger saved to", OUTPUT_DIR / "results_ledger.json")
```

Code A51. Notebook code cell 51

Source: COM7022 - Assignment/output/screenshots/appendix/cell51.png

AI use declaration. AI tools were used to support structuring and proofreading. All analysis was executed, verified and interpreted by the author, and every reported figure was checked against the notebook output. *(Align wording with the Arden AI policy before submission.)*